



M362 Unit 9

UNDERGRADUATE COMPUTING

Developing concurrent distributed systems



The client tier

Unit 9

This publication forms part of an Open University course M362 *Developing concurrent distributed systems*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Copyright © 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by SR Nova Pvt. Ltd, Bangalore, India.

Printed and bound in the United Kingdom by Martins the Printers, Berwick-upon-Tweed.

ISBN 978 0 7492 1597 2

1.1

CONTENTS

1	Introduction	6
1.1	The aims of this unit	7
2	Thin client	8
2.1	HTTP requests	8
2.2	HTTP command syntax	8
2.3	HTTP <code>GET</code> request parameters	10
2.4	HTTP <code>POST</code> request parameters	12
2.5	Session and connection issues	13
2.6	Problems with the thin client	16
3	Applet clients	18
3.1	Introduction – how applets work	18
3.2	How to write an applet	19
3.3	The applet lifecycle	22
3.4	Applet security	24
3.5	Problems with applets	24
3.6	Communication between applets and the other tiers	25
4	Application clients	28
4.1	Implementing an application client	28
4.2	Application clients without a client container	30
4.3	Deploying the application client	31
5	Push and pull	33
5.1	Introduction to push	33
5.2	Alternatives to push technology	34
5.3	Push using HTML header	36
5.4	Other push technologies	39
5.5	Push using UDP	39
5.6	Push using UDP multicast	40
5.7	Push using TCP/IP	42
5.8	Push using a messaging service	43

Continued over page

6	Time in distributed systems	46
6.1	The problem of time in computer systems	46
6.2	Perfect time	47
6.3	Synchronising across a distributed system	48
6.4	Logical time	52
6.5	An example – auction time	53
7	Case study	55
8	Summary	56
	Glossary	59
	References	62
	Acknowledgements	63
	Index	64



M362 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Chair, author and academic editor

Janet van der Linden

Authors

Anton Dil

Brendan Quinn

Michel Wermelinger

Critical readers and testers

Henryk Krajinski

Barbara Segal

Mark Thomas

Richard Walker

Yijun Yu

External assessor

Aad van Moorsel, Newcastle University

Software development

Ivan Dunn, Consultant

Course management

Linda Landsberg

Carrie Lewis

Barbara Poniatowska

Julia White

Media development staff

Ian Blackham, Editor

Sarah Gamman, Contracts Executive

Jennifer Harding, Editor

Phillip Howe, Media Assistant

Martin Keeling, Media Assistant

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Sue Stavert, Technical Testing Team

Andrew Whitehead, Designer and Graphic Artist

Thanks are due to the Desktop Publishing Unit of the Faculty of Mathematics, Computing and Technology.

1

Introduction

This unit is about the client tier of a Java EE system.

The client software may be a standard web browser. It may be more complex, involving **applets**, which, as we saw in *Unit 8*, are Java programs that are downloaded along with a web page and run by the browser. The client could also be an **application**, that is, a Java program that can run independently of a browser. A simple client with very limited functionality, mainly limited to display and communication, is known as a **thin client**.

More complex client software that carries out significant processing on the client machine is known as a **thick client** or a **fat client**. The more flattering term **rich client** is also used. A standard web browser is an example of a thin client, whereas a thick client is likely to be implemented as an application or possibly using an applet.

The client software may execute on a wide range of platforms – laptops, desktops, PDAs or mobile phones. It may be programmed in Java or in many other languages – as long as it uses an appropriate protocol for communicating with the **middle tier**. The client software may be inside a firewall that protects a system from external attacks, and it may itself have to communicate across a firewall on the server side. The communications link may be fast, reliable and high bandwidth, such as a high-speed LAN; or it may be an unreliable, low bandwidth, wireless connection. All these issues influence the communication protocols that can be used between the client and the server.

In earlier units we saw a number of low-level approaches to developing clients. For example in the *Unit 1* activities, the very simple Java program accessing a disk file containing the Music Store stock list could be considered a sort of client if the stock list file was stored on a remote machine. In *Unit 5* we demonstrated the use of Java sockets for a more sophisticated approach to client-server systems. More complex frameworks such as RMI and Java EE typically build on these lower-level approaches, but the user of higher-level frameworks need not be aware of this.

Within Java EE, the simplest approach to client software is a standard browser communicating with the web tier by means of the HTTP protocol. We will see that this is often satisfactory but that it also has a number of limitations. There are a number of more sophisticated approaches that can be used to address these limitations and this unit will consider some of these alternatives.

It is also possible that the client is not a system directly controlled by a human user, but is actually another program. This would mean that the server part of the Java EE system is providing a service to other applications running elsewhere on the Web. For this to work effectively there must be agreed standards for how such **web services** are described and accessed – suitable standards have been defined in recent years under the aegis of W3C, the World Wide Web Consortium. Web services are discussed in *Unit 11*.



W3C web services activity.

1.1 The aims of this unit

This unit will discuss:

- ▶ the various types of client software in a Java EE system;
- ▶ the options for clients in communicating with the Java EE server;
- ▶ the issue of time in distributed systems;
- ▶ how these concepts apply to our example system for The Music Store.

This unit involves reading and practical activities related to client tier concepts. We will finish the unit by adding an auction facility to our case study of The Music Store.

2 Thin client

The simplest form of client software for a Java EE application is a standard web browser that communicates with the middle tier by means of the HTTP protocol. In this case the middle tier comprises both a web tier and a business tier, with the web tier handling the HTTP requests and responses, as we saw in the previous unit. In this section we look at that interaction mostly from the client perspective.

2.1 HTTP requests

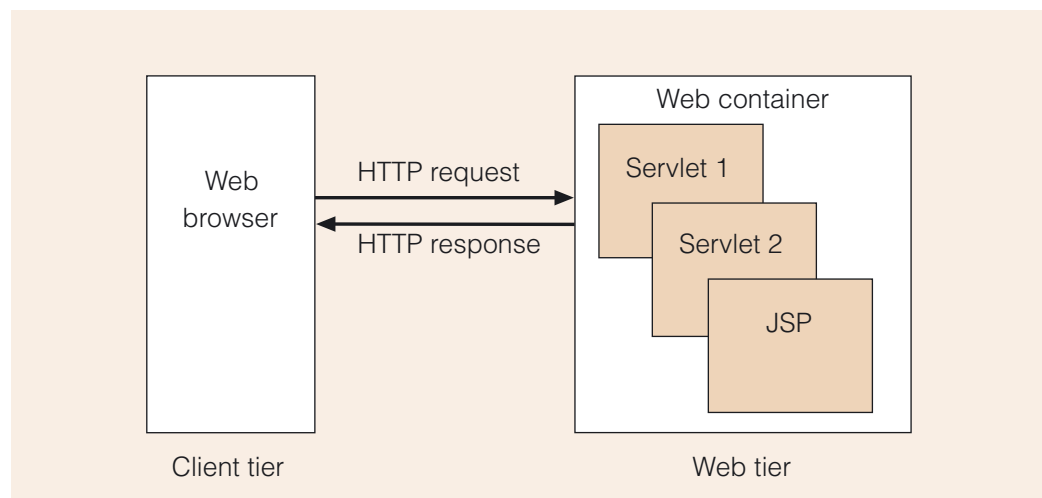


Figure 1 Web container running cooperating servlets and JSP pages only

In *Unit 8* we saw that the web tier is typically made up of a number of components, such as Java servlets and JSP pages that cooperate to process requests and send an appropriate response to the client. We now revisit a number of the examples from that unit to consider in more detail what happens in the client tier when such an interaction takes place.

The two most commonly used HTTP methods are `GET` and `POST`. You may recall that `GET` is normally used for requests that do not change the state of the system data, such as retrieving a static web page or querying a database. `POST` is normally used for requests that may change the state of the system data, such as updating a database. We have also seen that both `GET` and `POST` requests may have associated data, known as parameters. An important difference between `GET` and `POST` requests is the way that the parameter data is sent in the request. Before considering this, we give a brief reminder of the form of HTTP commands and responses as perceived by the client.

2.2 HTTP command syntax

HTTP requests consist of one or more lines of text. The first line takes the form of an HTTP method name, possibly followed by some additional information. When you click on a web link corresponding to another web page, this typically causes the browser to

send a GET request to the server. The following example shows the use of the GET method to request an HTML file called `index.htm` from a web server:

```
GET /index.htm HTTP/1.0
```

The 1.0 indicates that version 1.0 of HTTP is being used. If this request is successful, the server's response will typically start as follows.

```
HTTP/1.1 200 OK
```

This indicates that the server is using version 1.1 of HTTP (although it also understands version 1.0). The number 200 is a code to indicate to a computer that the request was successfully carried out, and the text `OK` tells a human reader the same thing. Any returned code of the form `2xx` (where `x` can be any decimal digit) indicates a successful request, and each code has a corresponding text explanation.

This response line is typically followed by **HTTP header** information, which includes the date, time, details of the server, the number of bytes sent and the type of content returned (normally text and HTML). Finally, the body of the response, consisting of the contents of the requested HTML file, is sent to the browser, which normally displays the file contents on the screen.

If the file requested is not present, or something else goes wrong, then the response will start with something different, such as:

```
HTTP/1.1 404 Not Found
```

In this case, code 404 indicates to the browser software that the file was not found. The corresponding text `Not Found` is for users who may be directly reading the HTTP response. In general, return codes of the form `4xx` and `5xx` indicate errors.

A POST request starts with a similar structure to the GET request, but is normally followed by parameter data, as we will see later. For example, the first line of a POST request could be as follows.

```
POST /servlet/MusicStoreLoginServlet HTTP/1.0
```

This indicates that some data is being sent for processing by the specified servlet.

2.3 HTTP GET request parameters

Let us consider how parameters are sent in a GET request. Recall the login screen for The Music Store which we discussed in the previous unit, reproduced below in Figure 2.

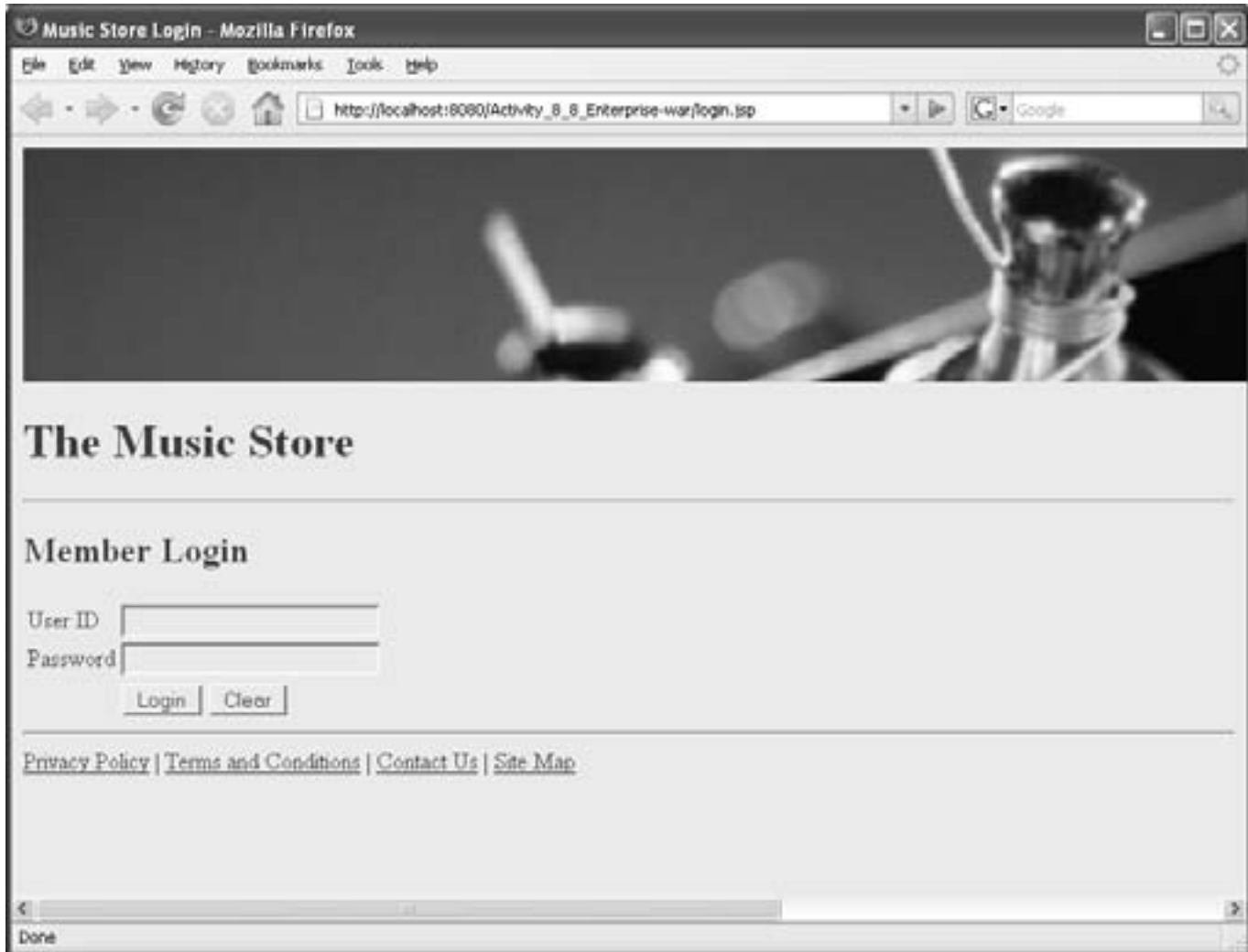


Figure 2 Example of a web form, used for logging in to The Music Store

Here is an extract from the HTML source for this web page, showing the HTML associated with the form part of the page.

```
<form method=GET enctype="application/x-www-form-urlencoded"
  action = "/servlet/MusicStoreLoginServlet">
  <p>
    <b>User ID </b>
    <INPUT TYPE="TEXT" NAME="userid" SIZE="23"> </p>
  <p>
    <b>Password </b>
    <INPUT TYPE="PASSWORD" NAME="password" SIZE="23"> </p>
  <p>
    <INPUT TYPE="SUBMIT" VALUE="Login">
    <INPUT TYPE="RESET" VALUE="Clear">
  </p>
</form>
```

When the user completes the form and clicks the **Login** button this causes a `GET` request to be sent to the server. Along with this request are the values entered for the user ID and password. Note the **attribute** within the `<form>` tag that specifies the encoding to be used for the parameter data, as follows.

```
enctype="application/x-www-form-urlencoded"
```

The parameter values for a `GET` request are sent by adding them on to the end of the destination URL. For example, if the user enters the value `Music1` for the user ID and `secret` for the password, the extended URL looks like this:

```
http://www.M362musicstore.com?userid=Music1&password=secret
```

This shows the URL followed by a question mark, to which is appended the list of parameters. The parameter names are paired with the values, such as `userid=Music1`, and each pair is separated by an ampersand (&) from any other name–value pairs. Note that there are no quotes around the parameter values.

The parameter names and values are also subjected to **URL encoding** to ensure that there is no confusion with **reserved characters** that have a special meaning in a URL, such as `/` or `#`. If a parameter name or value contains one of the reserved characters, then it is replaced by a percent-encoded character such as `%2F` for `/`, or `%23` for `#`. See the box below for more details. URL encoding can be rather complex – some of these complexities are explained in the box below, but you do not need to remember these details.

Complexities of URL encoding

There are some complexities in applying URL encoding to extended URLs. In particular there is a need to avoid confusion when the parameter names or values include **unsafe characters** – these include reserved characters, i.e. those that have a special role in a URL, such as forward slash (`/`), question mark (`?`), colon (`:`) and so on. The only characters that can safely be included in a URL, extended in this way, are the simple alphanumerics and the so-called mark characters such as `_`, `.`, `!`, `~`, `*`, `'`, `(`, and `)`.

To overcome this limitation we use encoding (hence the name URL encoding). Potentially unsafe characters can be included by encoding them using the corresponding ASCII code. For example, if the user entered the value `Music/Mad` for the user ID and `secret:X` for the password, then the encoded URL would be as follows.

```
http://www.M362musicstore.com?userid=Music%2FMad&password=secret%3AX
```

Here the forward slash in the user data is encoded as `%2F`, where `2F` is the hexadecimal ASCII code for a forward slash. A similar technique is used to encode the colon, whose ASCII code is `3A` hexadecimal. Other potentially unsafe characters such as spaces, quotes, ampersands and so on can be treated in the same way.

For historical reasons, spaces are normally treated differently – they can also be encoded by replacing each space by a plus sign. For example, if the user entered the parameter values `Music Mad` for the user ID and `secret X` for the password, then the encoded URL would be as follows.

```
http://www.M362musicstore.com?userid=Music+Mad&password=secret+X
```



URL syntax.

It would also be acceptable to encode this URL, using the ASCII code for space (20 hexadecimal), as follows.

```
http://www.M362musicstore.com?userid=Music%20Mad&password=secret%20X
```

Some browsers (such as versions of Internet Explorer) are lax in their treatment of spaces and may include them unencoded in the URL – this can lead to compatibility problems as it does not follow the relevant web standards, such as internet RFC 2396.

To include a character such as & or + or % that has a special use in URL encoding and extended URLs, it must also be percent-encoded. For example, a parameter with the unlikely name of A&M+5% would be encoded as A%24M%2B5%25, using the ASCII hexadecimal code values 24, 2B and 25 respectively for &, + and %.

Note that URL encoding is a general term which refers to the method of coding data regardless of whether this data is used to construct extended URLs. It is used in other circumstances, such as the data sent in POST requests, where the encoding has nothing to do with URLs – we discuss this in the next subsection.

This means of passing parameter values with an extended URL is quite simple but has a number of rather serious drawbacks, as follows.

- ▶ First, it makes the values of any data transmitted rather obvious to anyone looking over the user's shoulder or intercepting the GET request as it travels across the internet. The browser will not display the password on-screen as you type it in, but it would then send it in plain text as part of the URL encoding.
- ▶ The second problem is that, in some cases, adding the parameter data to the URL may make a very long URL string and most systems have a limit to the maximum length of the URL. This means that a request with a lot of associated parameter data cannot be sent as a GET request, even if this is appropriate in terms of its function. In such cases, we must use a POST request instead.
- ▶ The GET method is limited to data values that use ASCII characters. Only the POST method is specified to cover the entire (ISO 10646) character set.

Finally, we note that URL encoding is the only option for form data accompanying GET requests, so that the following content-type attribute shown above was not actually necessary:

```
enctype="application/x-www-form-urlencoded"
```

We included it above as a prelude to explaining URL encoding and in case you encounter it elsewhere. For example, as we will see, POST requests can have other content types, but they default to using URL encoding, so usually we can omit this attribute from the HTML for POST requests also.

2.4 HTTP POST request parameters

The POST request sends parameter data in name–value pairs as for the GET request, but this data is not added to the URL. Instead, the HTTP command line is followed by various items of header data, which in turn are followed by a series of parameter name–value pairs. Note that the name–value pairs are URL encoded.

For example, the login request sent by the client using a POST method might look as follows.

```
POST /servlet/MusicStoreLoginServlet HTTP/1.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 29
userid=Music1&password=secret
```

Here `MusicStoreLoginServlet` is the name of the Java servlet that will read the parameter data and process it – in this case by checking that the login details are correct, and allowing further access to the site if they are.

This approach allows large amounts of data to be sent as parameters, since it is not constrained by the maximum length of a URL. It also means that the parameter values do not appear on the browser screen as they would for a `GET` request.

This is still not ideal for transmitting sensitive information like passwords or credit-card numbers. For security-sensitive information, access should be carried out using the **HTTPS** (HTTP secure) protocol – this ensures that all information sent between client and server is encrypted. HTTPS will be discussed in *Unit 10*, which is on security.

The `POST` request is better than the `GET` request if there is a large amount of associated parameter data. It also has an alternative content type that should be used for submitting form data consisting of entire files, non-ASCII data or binary data. This is specified using the `enctype` attribute we saw above, but with the following value.

```
enctype="multipart/form-data"
```

The details of how to use this content type are outside the scope of this course.

As we saw in *Unit 8*, servlets or JSP pages can access the request parameters in exactly the same way, regardless of whether they were sent with a `GET` or a `POST` request. The underlying differences are dealt with by the Java libraries for HTTP.

Recall from *Unit 8* that `GET` requests should normally be used for operations that do not change data on the server.

2.5 Session and connection issues

In earlier units we met the idea of a session – a conversation between the client and the server, potentially consisting of a series of requests and responses. We also saw that, because HTTP is stateless, we have to take special care if we need to maintain information throughout a session. The example of an ecommerce shopping cart illustrates the problem, and the use of cookies with a session ID was a possible solution discussed in *Unit 8* (see Figure 3).

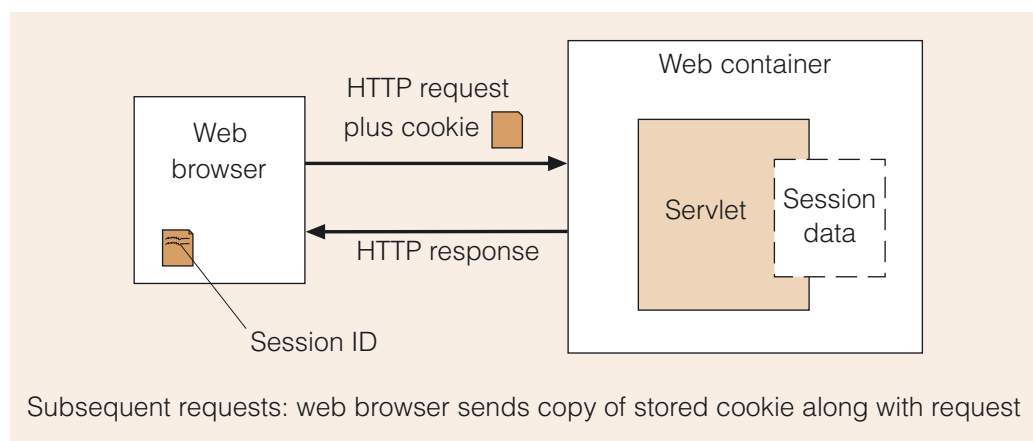


Figure 3 Using a cookie to maintain session information (extract from *Unit 8*, Figure 10)

We note that this approach requires no special action by the client. The browser automatically stores the cookie containing the session ID which the server sends with its first response. The client then sends the stored cookie back to the server with any subsequent requests, as shown in Figure 3. This behaviour is all part of the HTTP protocol – the cookie is sent as part of the header information that the client receives before the detailed content of any web page it has requested.

In this section we discuss another possible way of maintaining session state – **hidden form fields**. This is really a variation on the cookie idea. Data which needs to persist across more than one request is added to the HTML form by the server, and then sent back to the client. This data is formatted similarly to data which will be returned as a parameter (like the `userid` and `password` parameters above). However, these fields are marked with the attribute `HIDDEN`, which means that they are not displayed by the browser and thus the user will normally not be aware of their existence. For example, the login details (user ID and password) entered in the login screen might be sent back as hidden fields in the next form displayed to the user. (Figure 4 shows the sequence of requests and responses for a scenario where a user logs in and then is sent a form requesting information about discount vouchers.)

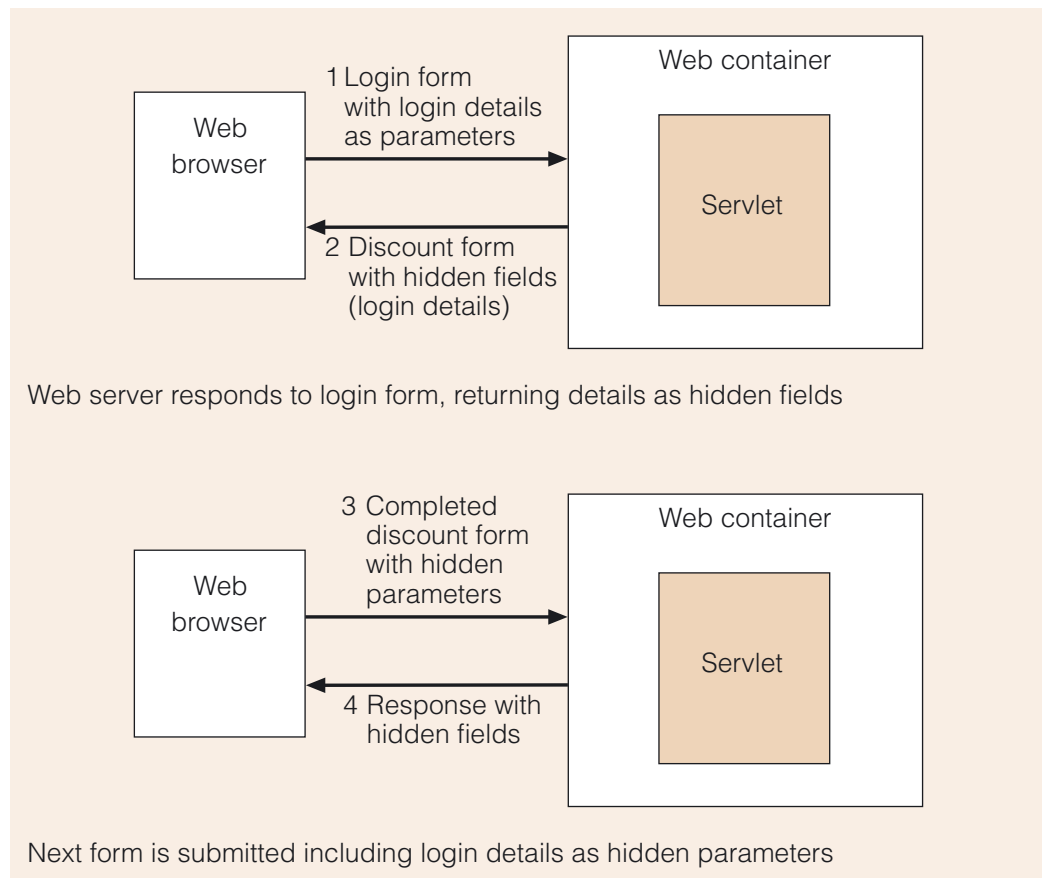


Figure 4 Using hidden form fields to maintain session information

For example, the next form might request the user to enter details of any discount voucher they may hold. The HTML code for the form section of this web page is as follows.

```
<form method=POST action =
    "/servlet/MusicStore/MusicStoreDiscountVoucher">
  <p>
    Discount Voucher Code
    <INPUT TYPE="NUMERIC" NAME="discountcode" SIZE="20">
  </p>
  <INPUT TYPE="HIDDEN" NAME="userid" VALUE="Music1">
  <INPUT TYPE="HIDDEN" NAME="password" VALUE="secret">
  <p>
    <INPUT TYPE="SUBMIT" VALUE="Submit">
  </p>
</form>
```

Note that as well as a conventional numeric field called `discountcode`, there are two hidden fields containing the values previously entered by the user when logging in. Figure 5 shows how this form might be displayed in a browser.



Music Store Discount Voucher - Mozilla Firefox

File Edit View History Bookmarks Tools Help

← → ↻ 🏠 📄 🔍 Google



The Music Store

Discount Voucher

If you have a discount voucher please enter the voucher code below

If not, just click on the **Submit** button to proceed

Discount Voucher Code

[Privacy Policy](#) | [Terms and Conditions](#) | [Contact Us](#) | [Site Map](#)

Done

Figure 5 Example of a discount voucher form with hidden fields, as displayed in a browser

There is no evidence of the two hidden fields here, but, when the form is submitted, the data for all three fields will be sent to the server as parameters.

In the above example, we have assumed that all the session data is repeatedly sent between the client and server as hidden fields. Alternatively, we could send just a session ID as a hidden field and use this to access more extensive session data stored on the server. This is similar to the approach used with cookies in Figure 3, and it avoids the obvious security risk of repeatedly sending passwords or other confidential data as hidden fields. It is also more efficient if large amounts of session data are involved, and may also make partial recovery of session data easier in the event of a system crash, since most of the session data will be stored on the server.

We note in passing that not all internet protocols are stateless. We can contrast HTTP with, for example, FTP, the internet file transfer protocol, which does maintain state information during a file transfer session.

TCP is the connection-oriented internet transport protocol.

In addition to being a stateless protocol, there is another way in which HTTP seems designed only for brief interactions. Each request under HTTP version 1.0 (defined in 1996) causes a new **Transmission Control Protocol (TCP) connection** to be established between the client and the server. This connection is automatically closed after the server responds. Any subsequent requests must again open a connection which will be closed following the response. Establishing a connection is a significant overhead, requiring the exchange of several data packets between the hosts – hence, for a session involving many requests, this is wasteful and tends to slow things down. For this reason, the later version of the protocol, HTTP v1.1 (defined in 1999) uses **persistent connections** – it normally keeps the TCP connection open for possible subsequent requests. This makes the v1.1 protocol exchanges a little more complex – for example, either the server or the client can close down the connection when it is no longer required – but communication should be more efficient.

As often on the Web, and elsewhere, good ideas are not universally adopted quickly, if at all. Not all web servers can deal with HTTP v1.1 requests, but those which do must also offer backwards compatibility – that is, they will also deal correctly with HTTP v1.0 requests.

2.6 Problems with the thin client

There are three main reasons why a standard browser client may not be sufficient.

First, the browser interface is rather limited. A standard web page consists of text, images and, potentially, the various interactive features offered by HTML forms. These interactive features include most of the standard user interface items, such as text fields, dropdown lists, command buttons, radio buttons and so on. While these are sufficient for many applications, sometimes more flexible or sophisticated interfaces are required.

Second, it is sometimes possible to improve the responsiveness of the system by carrying out some processing on the client, rather than doing everything on the server. This can eliminate network delays and may avoid some of the problems of dealing with heavily loaded servers.

Some of this additional flexibility or responsiveness can be provided by scripting languages such as JavaScript which run within the browser environment on the client. This can allow, for example, checking the validity of user data before sending it to the server. However, JavaScript, in common with most scripting languages, is somewhat limited in its capabilities compared with a full programming language like Java. Although Java and JavaScript share some of their name and syntax, they are not really closely related languages. There are also problems with some lack of standardisation of JavaScript across different browsers. A web development technique called AJAX (Asynchronous JavaScript and XML) has recently become a popular way to make web pages more responsive. AJAX allows the client and server to exchange small amounts of data without requiring reloading of a complete web page.



AJAX.

To allow more complex processing or display in the client than a standard browser with or without JavaScript, we must either use Java applets running in the browser, or else develop the client as a Java application, which can run independently of any browser. We consider each of these options later, in Sections 3 and 4 respectively.

The third deficiency is that a browser works with a request–response protocol only. We can say that it uses a **pull technology** – information travels from the server to the client only when the client does something to ‘pull’ that information to it. This means that the client does not necessarily know if something important on the server side has changed. For example, in The Music Store, it might sometimes be useful for the client to be informed of changes in the stock database, prices, the state of bids in an auction and so on. If the server is able to send information to one or more clients to convey information that is not in immediate response to a client request, this is known as **push technology**. This is discussed in Section 5.

SAQ 1

Comparing how parameter data associated with HTTP `GET` requests and `POST` requests is sent and received, answer the following questions.

- (a) What are the main similarities between `GET` and `POST`?
- (b) What are the main differences between `GET` and `POST`?

ANSWER.....

- (a) Parameter data is extracted by servlet code or JSP directives in exactly the same way, regardless of whether it is associated with a `POST` or a `GET` request. The underlying differences are hidden by the standard Java method calls or JSP directives. For both types of request, parameter data is normally sent in URL-encoded form, which means that any unsafe characters are replaced by their percent-encoded hexadecimal ASCII code values.
 - (b) `GET` requests append the parameter names and their values to the URL in the `GET` command line itself. `POST` requests send the parameter data in the body of the request, following the lines containing the `POST` command and any lines of header data. This means that parameters associated with a `POST` command are less visible to users. This can also accommodate much more parameter data, since the maximum length of the extended URL used by `GET` requests is limited on most systems. `POST` can also deal with more complex data such as files or binary data, whereas `GET` request data is much more limited in type.
-

3

Applet clients

We briefly introduced the idea of applets in Section 2 of *Unit 8*, in contrast with servlets, but we deferred a more detailed explanation to this unit, as they are part of the client tier. We assume that you have some idea from previous study or experience about what applets are and how they work, so we only briefly recap on these issues here. Some of the explanation in this section is summarised from other OU courses that explain applets in more detail, such as M254 or M257. You might like to consult those courses or other sources if you are not at all familiar with applets, although probably most of what you need to know is summarised in this section.

Here we will discuss how applets can be used in the client tier and how they communicate with the middle tier. Furthermore we will outline problems that may arise in using applets.

3.1 Introduction – how applets work

Applets are Java programs that are downloaded as bytecode along with a web page and run by the browser when the web page is viewed. The designers of Java deliberately restricted what applets are normally allowed to do – for example, applets usually cannot access files on the client computer where they are executed. This is because, generally, you should not trust programs that you download from the Web, perhaps unknowingly by simply clicking on a web link.

In many ways, a Java applet is similar to a Java application. In both cases, Java source code is compiled to bytecode and stored in a file with a name ending in `.class`. The difference lies in the way this bytecode file is then invoked and executed.

An application is typically installed directly on a particular computer and run there; although, as we will see later, it may be invoked from a remote computer. When an application is run, the bytecode is interpreted and executed by the Java Virtual Machine (JVM) on the computer where the application resides.

By contrast, the bytecode for an applet is normally run when an associated web page is loaded into a browser. If the web page is loaded from a web server on a remote computer, then any applet bytecode linked to that web page is also downloaded and run. The applet bytecode is actually interpreted and executed by a suitable web browser – a so-called Java-enabled browser. The sequence of events in the process of downloading and executing an applet is illustrated in Figure 6 (which reproduces Figure 1 from *Unit 8*).

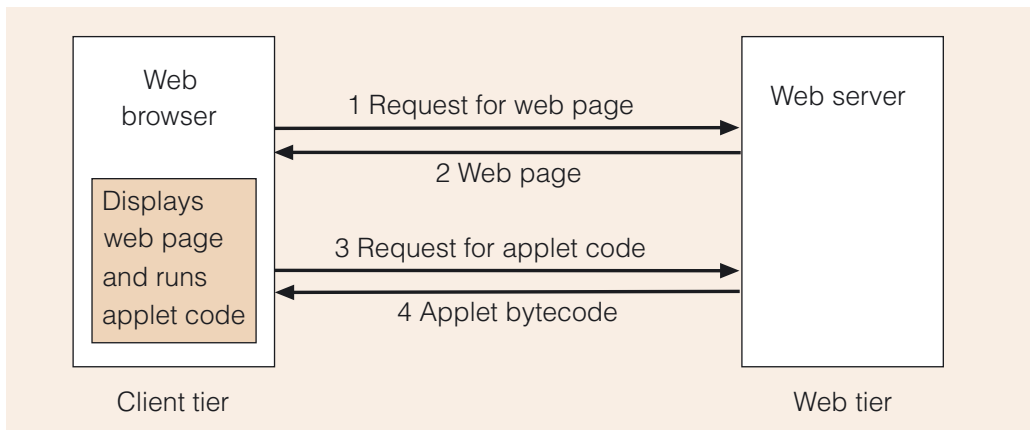


Figure 6 How applets are downloaded and executed – numbers show the order of events

Applets can be used for a wide variety of functions such as complex graphical displays, running simple games or processing data before sending it to the server. As the name 'applet' suggests, an applet is typically a small piece of code, but it need not be, as long as users are prepared to wait for a larger applet to download along with its associated web page. In most cases, users will not want to wait, so limiting applets to a manageable size is a good aim.

For testing an applet during development, there is also a standard application called the Applet Viewer – part of the JDK, the freely available Java Development Kit.

3.2 How to write an applet

In order to write an applet, it is necessary to first define a class that extends the `Applet` class from the `java.applet` package, or its subclass `JApplet` from the `javax.swing` package.

You will be pleased to know that much of what you should already know about writing applications also applies to writing applets. Applets can use almost all the Java language features and most of the standard API components. There are a few key differences, however, between applications and applets because applets must run in conjunction with a web page. These differences are as follows.

- ▶ An applet does not have a main method. It has an `init` method, which, to some extent, performs a similar role and which is invoked by the browser when the web page is loaded.
- ▶ Applets may only have a graphical user interface, and use normal Swing or Abstract Windowing Toolkit (AWT) features for this.
- ▶ When running in a browser, applets cannot use the standard streams such as `System.out` for input and output in the browser window.
- ▶ For the applet GUI, you do not need to construct a `Frame` or `JFrame` object as the browser window is used instead.
- ▶ An application with a GUI may use the `setSize`, `setTitle` or `setVisible` methods of the `Frame` class. For applets, sizing is done in the HTML file; they cannot have title bars and are made visible automatically.

This means that it is quite straightforward to convert a simple graphical application into an applet. The next subsection illustrates this with a very simple applet.

Applet Example

Here is a simple applet that displays a label containing a welcome message.

```
import java.awt.*;
import javax.swing.*;

public class MusicStoreWelcomeApplet extends JApplet
{
    public void init()
    {
        Container pane = getContentPane();
        JLabel label = new JLabel("Welcome to The Music Store!");
        pane.add(label);
    }
}
```

The applet is a subclass of `JApplet` from the Swing library. It defines one method, `init`, which is automatically run when the applet is first loaded, such as when you first view its web page.

Before we can run this simple welcome applet, we need an HTML page. This can contain any normal HTML tags and other content, but must also contain a special tag that links the web page to our applet. Traditionally, this is the role of the `<APPLET>` tag. Here is some suitable HTML.

```
<HTML>
<HEAD>
  <TITLE>Testing the Applet MusicStoreWelcomeApplet</TITLE>
</HEAD>
<BODY>
  <H1>The Music Store</H1>
  <HR>
  <APPLET CODE="MusicStoreWelcomeApplet.class" WIDTH=400 HEIGHT=300>
</APPLET>
  <HR>
  © The Music Store 2020
</BODY>
</HTML>
```

This contains a few items of standard content – a title for the web page, a piece of text that is displayed in large font, and two `<HR>` tags that cause the applet to be enclosed by two horizontal lines (horizontal ‘rules’) and a (fake) copyright statement. The `<APPLET>` tag has three parameters here – specifying the filename of the compiled code for the applet, together with the width and height in pixels of the applet window on the screen.

To run the applet, we can use a Java-enabled browser or the Applet Viewer program. We supply the name of the HTML file (not the applet file) to the Applet Viewer and it displays just the applet window, ignoring the rest of the HTML. The result should be similar to that in Figure 7.



Figure 7 Running the welcome applet using the Applet Viewer program

Using the Applet Viewer program allows you to test the applet independently of any web page it is to be embedded in. Using a browser to view the applet gives quite a different result, as shown in Figure 8.



Figure 8 Viewing the web page and the welcome applet using a browser

Now we see the effect of the HTML tags and other content of the web page. The applet is shown in a sub-window of size 400 by 300 pixels, as specified in the `<APPLET>` tag, and bounded by the horizontal lines specified by the `<HR>` tags.

This reinforces the point made earlier – the Applet Viewer is very useful for checking the applet by itself during development. For final testing of how the applet will look when its web page is displayed, you must use a browser. If an applet is to be made widely available, it should be tested using different browsers and browser versions, as there are quite a few differences in how browsers support various features of applets. We will discuss this in a later subsection.

HTML tag options for applets

In newer versions of HTML (since HTML 4.0), the `<APPLET>` tag has been deprecated and developers are expected to use the `<OBJECT>` tag instead. According to the

HTML 4.1 specification, the `<OBJECT>` element 'offers an all-purpose solution to generic object inclusion'. The intended advantages are as follows.

- ▶ It gives a means to include new and future media types.
- ▶ The `<APPLET>` element works only with Java-based applets.
- ▶ Current tags such as `<APPLET>` and `<IMG>` (for images) may pose accessibility problems.

We will see more about the `<OBJECT>` tag later, in Subsection 3.6.

Unfortunately, this specification has not been successful – not all browsers or versions of browsers support the use of the `<OBJECT>` tag in a consistent way, if they support it at all. The current situation is rather a mess. So we have to decide whether to comply with HTML 4 (and perhaps have to accept that our applets do not work on some browsers) or to continue to use `<APPLET>` tags, which are recognised on all browsers, apart from very ancient ones. The choice depends on how widespread you expect the use of the applet to be. If it is to be available only within a limited environment, such as an intranet, then you may be able to ensure that all the browsers can cope with `<OBJECT>` tags. In this unit, we will follow the approach recommended by Sun Microsystems and continue to use the `<APPLET>` tag to ensure wide compatibility across the internet.

3.3 The applet lifecycle

We have seen how to write a very simple applet, with only an `init` method. This method is invoked to initialise the applet when it is first loaded, normally when you first view the web page that links to the applet. In this sense, the `init` method acts like a constructor, and should be used for activities that need to be performed once in the lifetime of the applet. We would not normally put all the code for an applet into the `init` method.

For more complex applets, it is useful to have more control over whether the applet continues to run when the user moves the focus away from the related web page. The applet lifecycle methods, shown in Table 1, are useful in this case. It is important to note that these methods will be invoked by the browser or the Applet Viewer – they should not be invoked explicitly by the code for your applet. For example, when a web page with a linked applet is first viewed using a browser, the `init` method of the applet will be invoked, followed by the `start` method.

Table 1 Applet lifecycle methods

Method	Description
<code>init</code>	Invoked when the applet is loaded, e.g. when the applet web page is first viewed.
<code>start</code>	Invoked when the applet is made active, e.g. when the applet web page is viewed (possibly again).
<code>stop</code>	Invoked when the applet is made inactive, e.g. when the user moves away to view a different web page.
<code>destroy</code>	Invoked when, e.g. the user closes down the browser or the Applet Viewer.

The applet inherits the lifecycle methods, which are shown in Table 1, from its superclass `Applet`, and these methods should be overridden, if required. The superclass methods typically do nothing. Let us consider when we might need to override one or more of these methods.

The `init` method and the `destroy` method are each invoked once during the lifetime of the applet, but the `start` and `stop` methods may be invoked several times if the web

page is revisited. If an applet uses significant processor resource when running, such as might occur with a complex animation, it is good practice to implement `start` and `stop` methods. This ensures that the processor is being used only when the web page is viewed. The `init` method should be used for one-off initialisation, such as initialising instance variables or allocating resources (e.g. defining an array). The `destroy` method is less commonly required, especially since Java garbage collection deals with releasing memory resources. However, it should be used for any final clean-up operations or releasing of resources before the applet terminates. Figure 9 illustrates the sequence of events.

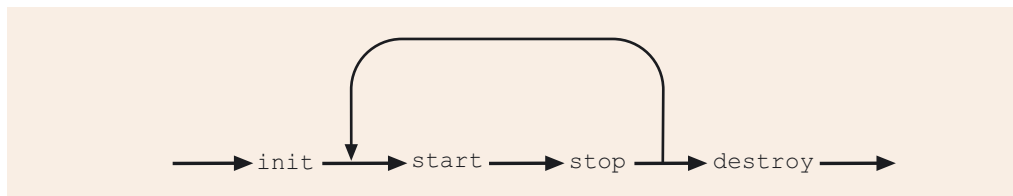


Figure 9 Applet lifecycle

For example, an applet that uses threads would typically use the `init` method to create the threads and the `destroy` method to finally remove them. The `start` and `stop` methods might start and suspend execution of the threads respectively.

There are a few other important methods for applets, and these are listed in Table 2.

Among these methods, the most frequently used is the `paint` method, which is invoked by the system to draw the applet window and any graphical content it may have. You may need to override the `paint` method that is inherited from the `Applet` class, in order to define the graphical content of your applet window. This is particularly useful if the graphical content is to be dynamic.

The other methods in the table are useful for checking whether the applet is still active, for changing the size of the applet window from the size originally defined in the HTML file, and for various multimedia operations such as retrieving images and audio files. Note that you do not have complete freedom to resize the applet window, since it has to fit within a maximum space defined by the `<APPLET>` tag in the HTML file. See the `Applet` class in the Java API for full details of these methods.

Table 2 Some other important applet methods

Method	Description
<code>paint</code>	Invoked to draw the contents of the applet window (executed when the applet window appears or is changed).
<code>isActive</code>	Returns true if the applet is active; an applet is marked active just before its <code>start</code> method is called and is marked inactive just before its <code>stop</code> method is called.
<code>resize</code>	Requests a new size (in pixels) for the applet window – use with caution for applets to be viewed using a browser, as the size specified by the <code><APPLET></code> tag defines a maximum size.
<code>getImage</code>	Retrieves an <code>Image</code> object from a specified URL; this image can then be displayed on-screen.
<code>getAudioClip</code>	Retrieves an <code>AudioClip</code> object from a specified URL; this audio clip can then be played.
<code>play</code>	Plays the <code>AudioClip</code> object to be found at a specified URL.

The abstract class `Image` is the superclass of all classes that represent graphical images. The `AudioClip` interface is a simple abstraction for playing a sound clip. See the API for details.

3.4 Applet security

Earlier, we mentioned that there are some restrictions on what applets are normally allowed to do. When these restrictions are in place, the applet is said to be running in the **sandbox**. This is necessary because applets are downloaded and run from web servers, which may be remote and of unknown trustworthiness, perhaps even by accidentally clicking on a web link. It is usually not possible for the user to prevent an applet from running. If applets had no security restrictions, then a badly written or deliberately malicious applet could do enormous damage on the computer that runs the applet.

The standard restrictions that apply to applets are as follows.

- ▶ They cannot read from or write to the local computer's file system.
- ▶ They cannot run any executable programs on the local computer.
- ▶ They cannot communicate with any computer other than the server from which the applet was downloaded – this is sometimes explained as 'applets can only phone home'.
- ▶ They can find out only very limited information about the local computer, such as the version of Java or the operating system in use.
- ▶ If an applet creates a pop-up window, the window displays a special warning message.

These restrictions can be enforced because the applet is run by the browser, and the code executed by the applet is monitored by software known as the Java Security Manager. It is possible to allow applets more freedom, typically when the applet comes from a known and trusted source, such as an intranet. This lifting of some or all security restrictions is done by a process known as signing the applet – this uses cryptography to identify the origin of the applet. We return to the topics of signed applets and security policies in *Unit 10*.

3.5 Problems with applets

We have seen that one of the potential advantages of applets is that they run in a browser, an environment that is widely available. This can also be a disadvantage because there are many different browsers. There are also quite a number of versions of the more popular browsers and they vary in their treatment of applets. The browser's Java Virtual Machine (JVM) which runs the applet has historically been provided as part of the browser, but without any strong standardisation. Hence it may be necessary to develop several different versions of applets and their supporting software to ensure that they run on most common browsers.

Fortunately, there is a way out of this unsatisfactory situation. Sun Microsystems offers freely downloadable software that can be 'plugged in' to most browsers to enable them to run applets using current versions of Java. The **Java Plug-in**, as it is called, provides a standard and up-to-date JVM and the Java Runtime Environment (JRE). Effectively, Sun Microsystems has solved the compatibility problems by taking back control of the browser JVM.

The word 'sandbox' is an American term for a children's sandy playing area (a 'sandpit' in UK terminology).

Some systems may come with the Java Plug-in already installed. It is a significant download, but the large download should be required only once, with possible occasional updates when you wish to take advantage of a new version of the Java language. On some platforms the Java Plug-in may be automatically installed when either the JRE or the Java Development Kit (JDK) is installed.

If you experience any problems in running applets from a browser, it may be useful to check whether the Java Plug-in is already installed on your computer. The details of how to do this vary between platforms, but are readily available on the Web.



Activity 9.1

Running an applet.



Java Plug-in technology.

3.6 Communication between applets and the other tiers

A web page generated by the web tier and sent to the client can include an embedded applet. In this case, the applet may communicate with web tier components on the server.

Applets from a JSP page

You can include an applet in a JSP page by using the `<jsp:plugin>` action. This can also be used to include a JavaBeans component of the type we saw in *Unit 8*, but we focus here only on its use with applets. The `<jsp:plugin>` element generates appropriate HTML (such as the `<OBJECT>` tag and associated parameters) to allow the applet to be downloaded and executed on the client. If necessary, it will also download the Java Plug-in software to enable the client to run the applet.

The following shows how the simple welcome applet introduced above in Subsection 3.2 might be embedded in a JSP page.

```
<jsp:plugin
  type="applet"
  code="MusicStoreWelcomeApplet.class"
  codebase="/applets"
  align="center" height="300" width="400"
>
  <jsp:params>
    <jsp:param name="Message" value="Sale now on" />
    <jsp:param name="DiscountRatePercent" value="10" />
  </jsp:params>
  <jsp:fallback> The Java Plug-in did not start </jsp:fallback>
</jsp:plugin>
```

The first three attributes, `type`, `code` and `codebase`, are compulsory. The `code` attribute specifies the filename for the compiled applet code (bytecode) and `codebase` indicates the location of the file. There is also a list of optional attributes – we show just three of the most commonly used: the alignment (left, centre, etc.), and the height and width of the applet in pixels – in general, these attributes correspond closely to the attributes of the `<APPLET>` tag and the `<OBJECT>` tag.

The optional `<jsp:params>` element specifies any parameters to the applet. Our welcome applet in Subsection 3.2 did not actually use any parameters, but these are included here just to show how it works – if there are no applet parameters the `<jsp:params>` element can be omitted.

The optional `<jsp:fallback>` element indicates the content to be displayed by the client browser if the plug-in cannot be started. If the plug-in starts but the applet does not, the plug-in itself usually displays a pop-up window explaining the error to the user.

In the example above, if the plug-in did not start, the following message would be displayed in the applet window:

The Java Plug-in did not start

Applets from a servlet

The code to invoke an applet can also be embedded within a Java servlet. This is done by directly sending the relevant HTML code (including `<APPLET>` or `<OBJECT>` tags) to the response stream. So, for example, to generate appropriate tags in a web page for the simple welcome applet as above, we would have statements in the servlet code like this:

```
out.println("<APPLET CODE=\"MusicStoreWelcomeApplet.class\"");  
out.println(" WIDTH=400 HEIGHT=300> ");  
out.println("</APPLET>");
```

Note the use of an escape character `\` for quotes within the string.

However, it is good practice to use JSP pages rather than servlets for constructing an HTTP response.

Browser compatibility problems

As we saw earlier in this section, client systems may need the Java Plug-in and possibly a security policy file to run the applet successfully.

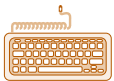
One of the supposed benefits of the `<OBJECT>` tag over the `<APPLET>` tag is that it can include a parameter indicating a URL from which the necessary plug-in can be downloaded if it is not already available on the client. However, we also noted earlier that support for the `<OBJECT>` tag is patchy and not well standardised. This means that if you are including `<OBJECT>` tags directly in a JSP page or in servlet output, it may be necessary to enclose them in complex conditional code to check which browser is in use, so as to follow different HTML instructions in each case.

The `<jsp:plugin>` tag is supposed to take care of these complications by generating appropriate code around the `<OBJECT>` tag to take account of browser variations. In practice, things are sufficiently complicated that neither the `<jsp:plugin>` tag nor the various recipes on the Web for complex conditional `<OBJECT>` tag code can be guaranteed to work in all cases.

So, as with standard web pages, to ensure that applets are likely to run in most cases, we often have to revert to using the simple `<APPLET>` tag in JSP pages. If the relevant plug-in is not installed on the client, then the `<APPLET>` tag will not download it, so it is necessary to arrange this in some other way, before the applet runs.

Applets and the business tier

If the applet is providing a comprehensive user interface, then it is possible to bypass or even dispense with the web tier and have the applet communicate directly with the EJB session beans in the business tier (Figure 10).



Activity 9.2

Running an applet client embedded in a JSP page.

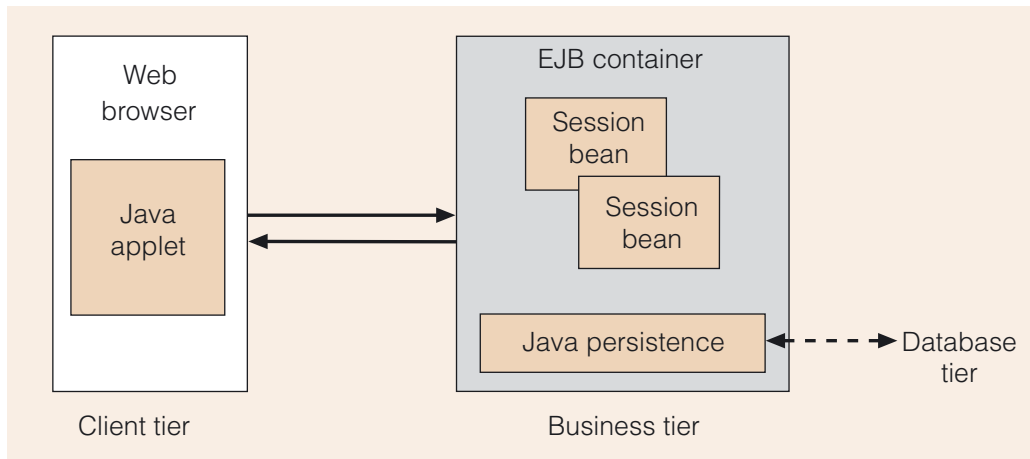


Figure 10 Applet client interacting directly with the business tier

There are a number of ways in which applets can achieve this communication. These include RMI, various socket-based approaches or messaging (for example, using JMS which was mentioned in *Unit 5*). Remember, however, that applets can only 'phone home' – they can communicate only with the server from which they were downloaded. We consider these options in more detail in Section 5, which also includes a number of practical activities.

4 Application clients

Instead of using applets to implement more flexible clients, it is also possible to use a Java application running on the client computer within its own JVM. This so-called **application client** communicates directly with the session bean components of the business tier.

To do this, the application client must be aware of the remote interface that defines the available methods of any session beans it uses, since the session bean method code actually runs on the server. In this section we will see how this communication is achieved.

4.1 Implementing an application client

The first stage of creating an application client for Java EE is similar to creating any standard Java application. Following the requirements for the functions and user interface of the client, the first stage is to design and code an appropriate set of Java classes and compile these in the normal way. This application can use any of the standard Java user interface classes from the Swing library to create a richer user interface than is normally possible with a web browser. Alternatively, you may wish to create a very simple user interface, perhaps to restrict what the user can do, or to suit a client device with restricted display resources or other limitations. The key thing is that all the facilities of the Java language on the client device can be used in application clients, unlike untrusted applets, for example, which, as we saw in *Unit 8*, have a number of restrictions.

There are two main ways for the application client to communicate with the business tier on the Java EE server. The first way is to run the application client in a special software environment, the **application client container**, as illustrated in Figure 11.

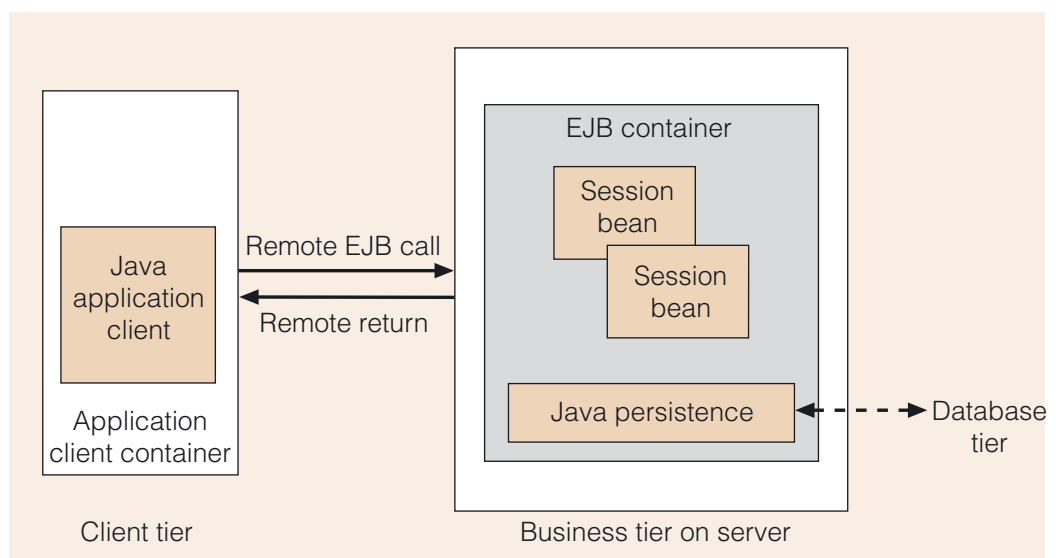


Figure 11 Application client using container facilities to access EJBs in the business tier

This container serves the same sort of function as the servlet container or the EJB container on the server – it provides software that takes care of many of the routine requirements of its components for communication, security and so on. In particular, an

application client can access remote EJBs running on the server without the programmer having to write any special code for lookup and communication.

With the client container approach, the client code uses annotations to access the interface of the remote EJBs. This is the same dependency injection approach that we encountered in the examples of session beans in *Unit 7*. We must use the remote interface since the client and server will be running on different machines (in any real system). For example, suppose we have a session bean that checks user login details, among other things, and has a remote business interface, part of which is as follows.

```
@Remote
public interface PasswordCheckerRemote
{
    void checkLoginData(String userid, String password);
    boolean isLoggedIn();
    ...
}
```

The implementation of this interface is by the `PasswordCheckerBean` class. The `PasswordCheckerBean` objects run on the server in an EJB container as we have seen previously. Assuming we have an application client running in an application client container on the client, then it can access the business interface of the session bean by dependency injection as follows.

```
@EJB
private static PasswordCheckerRemote passwordChecker;
```

This defines a local variable called `passwordChecker` on which we can invoke any methods of the session bean, just as for any Java class. For example:

```
if (passwordChecker.isLoggedIn())
{
    // Do something now you are logged in.
}
```

Dependency injection works here because we have an application client container that takes care of all the details of looking up remote objects and returning a suitable reference. The actual communication is carried out using RMI-IIOP, a variant of normal RMI, but since this is all handled by the container software, the programmer of the application client need not be aware of this. We will meet IIOP (Internet Inter-ORB Protocol) again in *Unit 11* when we discuss ORBs (object request brokers) and CORBA.

In some circumstances it is not appropriate or possible to use an application client container. In earlier versions of Java EE this client container was not available. Because of continual developments and enhancements to Java EE, the software support for client containers has sometimes been a moving target – that is, subject to rapid change. Finally, the client container software may be very demanding in resources, requiring a lot of memory, storage or processor time, making it a better idea to ‘do it yourself’, especially if the client has limited hardware resources.

The alternative way to link an application client with the server is to program the communication yourself, by looking up the EJB objects in a JNDI directory and communicating with them via their remote interfaces. This is explained next.



Activity 9.3

An application client running in a client container.

4.2 Application clients without a client container

JNDI stands for **Java Naming and Directory Interface** and was explained in *Unit 7*.

In this case, the application client software runs directly in a JVM on the client and must access the EJB session beans by using JNDI. The JNDI names for any session beans to be remotely accessed must be specified when building the EJB classes. This is normally done using tools provided by a suitable IDE for Java EE, or it can be done using annotations; for example, as follows for the `PasswordCheckerBean` class.

```
@Stateful(mappedName="Checker")
public class PasswordCheckerBean implements PasswordCheckerRemote
{
    ...
}
```

This declares a stateful session bean with the JNDI name `Checker`. Note that the bean implementation class is annotated, not the business interface. If you do not specify a JNDI name using annotations or otherwise, the system will use a default name which is the fully qualified name of the interface. It is usually better to define the JNDI name explicitly.

When the client wants to invoke an EJB session bean method it first looks up the session bean using its JNDI name. The directory service uses the JNDI name to provide a reference to the remote EJB object, as illustrated in Figure 12.

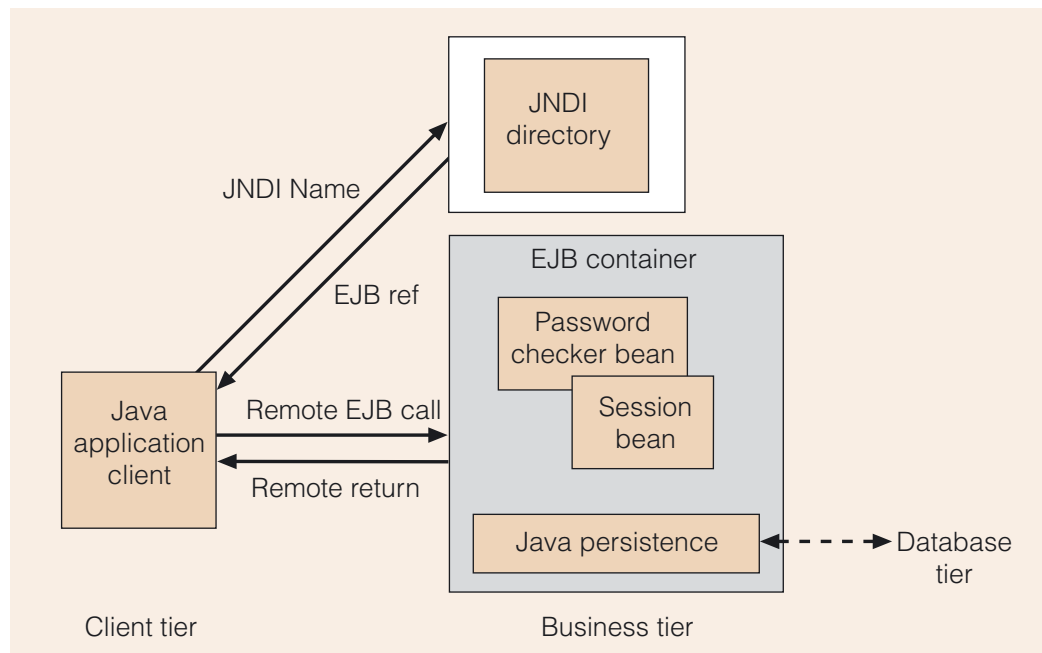


Figure 12 Application client using JNDI lookup to access EJB session beans in the business tier

In the diagram we show the JNDI directory running on the server. Typically it runs as part of the Java EE application server, but it would be possible to have the JNDI directory running on a different host. We will not consider this scenario further in this course and all our examples will have the JNDI directory hosted by the Java EE server.

Suitable code for this lookup process by the client is as follows.

```
PasswordCheckerRemote remoteBean = null;
try
{
    Context context = new InitialContext();
    remoteBean = (PasswordCheckerRemote) context.lookup("Checker");
}
catch (NamingException ex)
{
    System.out.println("Lookup failed: " + ex);
}
```

`Context` and `InitialContext` are classes defined in the package `javax.naming`, and these give access to the JNDI naming service. If the lookup is successful, the `remoteBean` variable is set to hold a reference to the remote EJB session bean. Note the need to cast the returned reference from the `lookup` method to be of type `PasswordCheckerRemote`, since `lookup` returns a reference to `Object`. Once the client has this reference, the session bean methods can be invoked directly as for the application client container.



Activity 9.4

Using an application client running in a JVM.

4.3 Deploying the application client

When using a thin client such as a standard browser, the software for a Java EE application need be installed (deployed) only on the server or servers for the system. We can normally assume that each client computer has its browser already installed. Even when using an applet client, the normal operation of browsers ensures that the applet will be downloaded to the client automatically as required.

When using an application client, we must consider a further step – ensuring that the application client is deployed on all client computers. This highlights one of the potential problems with application clients – any changes to the application client will require another operation to deploy the new client software to all client computers. In a distributed system with many clients this could be a major operation, with scope for problems if not all client machines are updated successfully.

The most basic approach is that the application client and any supporting software must be copied to each client host computer. Since we are discussing a distributed system, this is most conveniently done by downloading the application client software over a network link and then installing it on the client host. If this is a manual process or even an automated process there may be problems in ensuring that all clients have been updated – for example, some clients may connect only infrequently.

Note that Java EE is only a specification for enterprise software – there are a number of implementations provided by software vendors, such as IBM or Sun Microsystems, that comply with the Java EE specification. Some details of deploying application clients are not defined in the Java EE specification – they are implementation dependent. Some implementations of Java EE servers offer the ability to deploy the application client on the server and then have it automatically distributed and installed on client machines. This process may use the Java Web Start software, to retrieve and install the application client software. The application client is then stored on the client machine and further downloads are needed only in the event of software updates to the application client. In this way, the application client behaves somewhat like an applet – it is downloaded on demand – with the advantage that the download happens only once for each version of the software.



Java Web Start technology.

SAQ 2

What are the main issues to bear in mind when comparing thin clients, applet clients and application clients in Java EE?

ANSWER.....

Thin clients, typically browsers, may have limited functionality and operate mainly in pull mode, although these can be improved to some extent by the use of scripting languages such as JavaScript. They require only very limited hardware and software resources on the client machine.

Applets and application clients can provide a richer user interface and take some of the processing load away from the server for a more responsive client. They normally require more processing and storage on the client machine than thin clients.

Thin clients and applet-based clients have minimal deployment issues, assuming that most clients have a standard browser, though sometimes issues of browser versions and compatibility may arise. Compatibility issues are of more concern for applet-based clients since browsers vary in their treatment of applets, although this is often solved by using the Java Plug-in.

For application clients, there has to be a way of installing the client software on each client machine and keeping this up to date – this can be problematic, but some Java EE implementations allow deployment of the application client software on the server for automatic downloading to clients as required.

5

Push and pull

In this section we explain the idea of push technology and why it can be useful. We outline some alternatives that do some of what is required from push technology, and discuss why they are not entirely satisfactory. We also demonstrate a number of ways in which push technology can be implemented.

5.1 Introduction to push

The normal mode of operation for a web browser is pull technology – the client receives information or updates only in response to a request that it sends to the server. That is what we mean by describing HTTP as a request–response protocol. There have been various attempts over the years to develop push technology – where the server can notify its clients of any significant changes without the clients having to explicitly make a request each time. The potential advantage of this is that it automatically keeps clients up to date with the information on the server – this could be particularly important in fast-moving environments such as online auctions or stock markets.

Exercise 1

- (a) What requirements do you think a push technology approach might have?
- (b) Can you think of any push technology approaches currently in use on the Web?

Discussion.....

- (a) Since the server is to send out data without an explicit client request, there must be a way for the server to ascertain to which clients the data should be sent. One way to do this is for each client to send an initial message to the server to register their interest in receiving regular information updates from the server. It could be quite resource intensive if the server has to send out the same update many times to all its registered clients – so it would be good if there was a way to send the update message only once but have all interested clients receive it. You may have thought of some different or additional requirements.
- (b) The RSS (Really Simple Syndication) system, now available on many websites, arguably works in a push technology way. This allows you to sign up to receive occasional updates from a website whenever something important has changed or occurred. This avoids users having to frequently check a site to see if anything has happened. Only those clients who have registered for a given RSS service will receive the updates. However, there is some dispute about whether this is a pure push technology because to receive updates it is necessary to run an RSS reader program that polls the relevant website regularly while it is running. You might have thought of a different example – does it meet your requirements or our requirements from part (a)?

Clearly, unrestricted push technology could become rather like email spam – clients could be overwhelmed by frequent and possibly unwanted updates. This was more or less what happened to some of the early, rather hyped attempts at push technology.

PointCast push technology

The PointCast network was launched in 1996 with great 'dotcom' hype about being the first push technology. Here is a sample of the promotion at the time.

News addicts will jump for a new product from PointCast that turns your Windows screensaver into an up-to-the-minute news broadcast. Called the PointCast Network (PCN), the new program gets online news from such services as Reuters, the S&P stock ticker, PR Newswire, SportsTicker, AccuWeather, and Variety. When you're not using your PC, newsfeeds and ad banners scroll across your screen. You can click on items to access more information on news and other features that catch your eye.

PointCast covers national, international and top business news, offers a stock ticker and weather forecasts and maps, allows you to get the latest scores and analysis for all professional and some college sports, and provides you with the latest entertainment news, horoscope charts, and lottery numbers.

(Louis, 1996)

Initially it was wildly popular and the value of the company rose to hundreds of millions of dollars, attracting 1.5 million users, and reported takeover interest from many major media and software companies. However, disillusion soon set in. It was banned from many corporates because of 'excessive bandwidth' requirements. It was also far too much for the home dial-up systems that were the norm at that time. As a free download it was dependent on advertising and pushed numerous advertisements at unwilling users. The whole thing more or less died in 1999, by which time the company valuation had fallen to less than \$20 million dollars from its reported peak of \$450 million. The PointCast Network closed in 2000, poisoning the term 'push' as a business proposition for many years after.

The PointCast service was accessed from a screensaver client, not from a web browser, although later versions did appear as browser plug-ins. It has recently received the accolade from PC World magazine as one of the '25 worst tech products of all time'.

(Himelstein and Siklos, 1999; Wikipedia, 2007)

5.2 Alternatives to push technology

Before we consider push technology in more detail, it is useful to briefly look at and compare the alternatives. Suppose a user is employing a web browser running on the client to get information from a web server where the information changes frequently – for example, the latest bids in an online auction for a musical instrument.

To get regular updates from the server, the user could repeatedly click the browser refresh button, hence causing frequent reloading of the web page from the server. This process can be automated, by including special instructions in the HTML of the web page, to indicate to the browser that it should be reloaded at regular intervals. This is known as **client polling**. The necessary instructions take the form of a special HTML tag in the HTML document. For example, to set the client to refresh the web page every five seconds, the following tag could be used.

```
<META HTTP-EQUIV= "Refresh" CONTENT=5>
```

This META tag must be inside the HEAD section of the HTML document, so that it is processed before any text or images in the BODY section. Note also, that this tag actually

causes only *one* reload of the web page after five seconds, not an infinite series of reloads. Assuming that the reloaded page also contains the same `META` tag, this will trigger another refresh in five seconds and so on, until the user switches to viewing a different URL (see Figure 13).

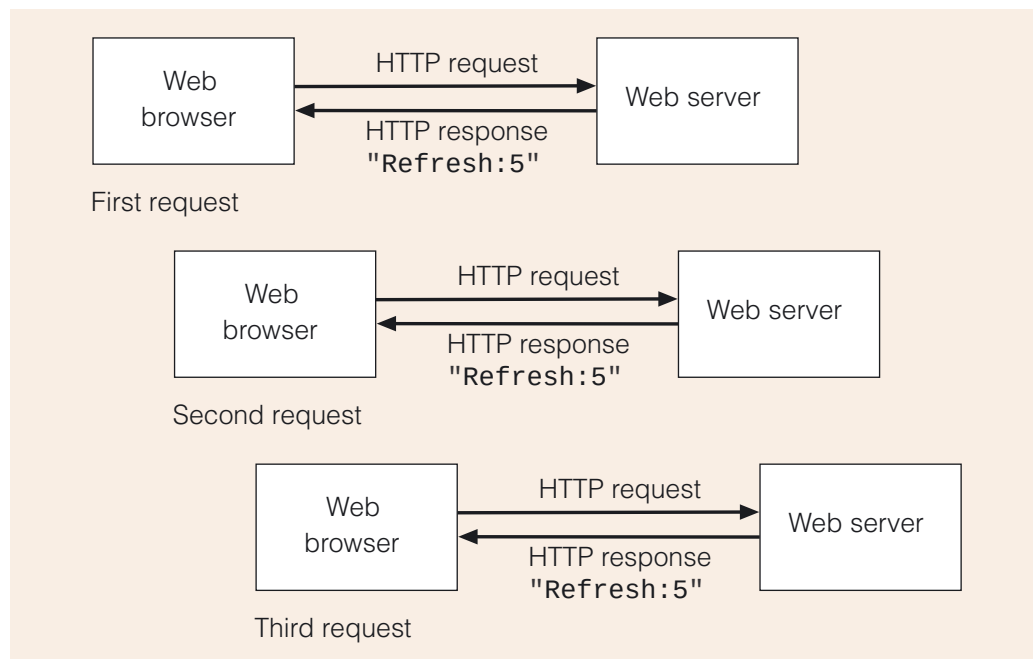


Figure 13 Client polling – client sends repeated HTTP requests for the same URL

Client polling is simple in that no special technology is required by the client or the server, but it, potentially, generates a large amount of internet traffic. Because of the way HTTP works, it normally also requires a new connection to be opened between the client and the server for each request, and this connection is then closed. Moreover, if the refresh interval is short compared with the frequency of new bids, most of the web page downloads will be completely unnecessary.

This is not necessarily a problem in a small-scale system, but in a large-scale online auction of any item there might be hundreds of people taking an interest. Multiply that by the many thousands of items for sale on major auction systems across numerous countries and you start to see the scale of unnecessary traffic that would be generated by using automated refresh. And that excludes the possible effect of many other online applications apart from auctions.

There are two aspects to fix:

- ▶ each client is sending separate update requests, each of which requires significant server resources to process;
- ▶ whole page updates are being sent by servers even when little or nothing on the web page has changed.

Clearly we are approaching things from the wrong end. Clients do not know when they need an update but the server does. Clients should 'express an interest' in receiving updates and should receive these only when the server knows that some significant change has occurred.

We can implement this, while still using a client polling approach. In fact this is how **RSS** works. Clients register with a server for a particular RSS feed by the user clicking an RSS feed button on the website of interest. This will download an XML file containing a summary of selected items on that website. Choosing a particular RSS feed allows the user to specify the events of interest to them – for example, it might be the current highest bid of an online auction. To check for updates, the user runs an RSS reader

program on the client machine – this accesses the relevant website and checks if any changes have occurred since the last update received. If nothing relevant to that feed has changed, then nothing fresh is downloaded, hence saving large amounts of data transfer. If something has changed, then the updated information is downloaded.

The RSS reader actually polls the server at regular intervals – this is more acceptable than automatically refreshing the whole web page by client polling because most of the time the only traffic consists of small RSS messages to and from the server, checking and confirming that nothing has changed (see Figure 14).

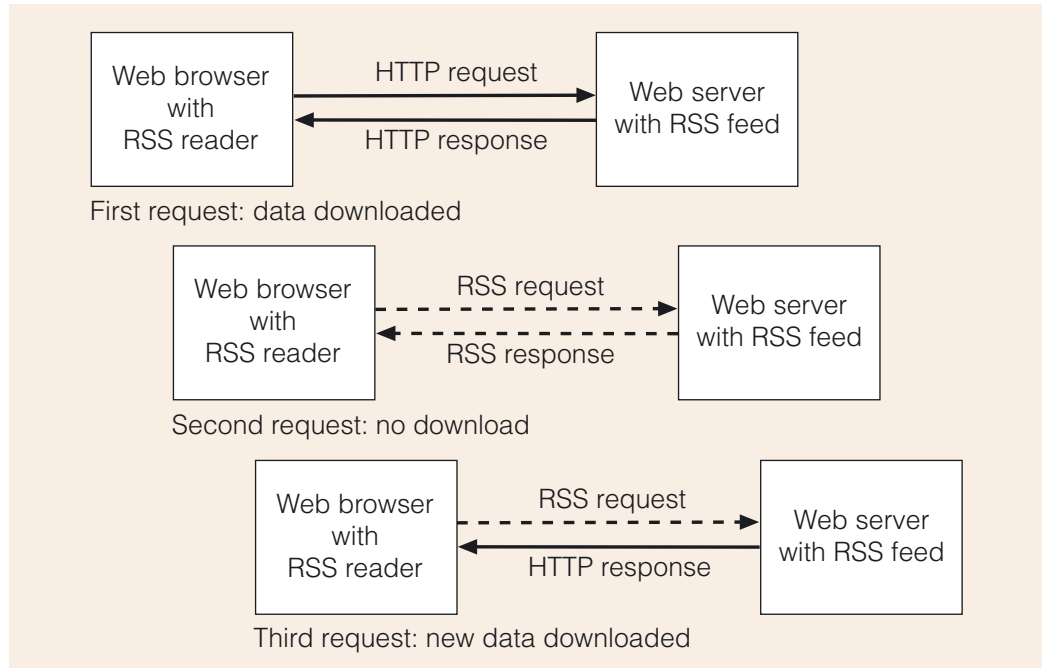


Figure 14 RSS – client receives new data only when there is something new

RSS and systems like this address the second of the two problems above – the overhead of web page updates when nothing has changed. They do little for the first issue of clients continually using server resources for repeated requests – for that we must look at push technology.

5.3 Push using HTML header

There is a quite simple way to implement server push using the `Content-type` attribute that forms part of the header information returned in an HTTP response. This can be configured to cause the server to keep open the connection to the client, even after it has sent a response. The server can then send another block of data (for example, text or images) which replaces the first block of data. This process can continue indefinitely, either at regular intervals or whenever the server has something new to send. The connection can be closed by the server at any time – the client can close the connection only when the user switches away from viewing that URL.

To explain how this works, we first consider the `Content-type` attribute of the HTTP response header. Normally this indicates the type of content to be found in the data that follows the header, such as text, image or HTML. For example, a typical web page containing text and HTML is indicated by a header field as follows.

```
Content-type: text/HTML
```

The server push is implemented using a content type called `multipart/x-mixed-replace`. This specifies that the page content will be delivered from the server as a series of 'parts', with each part replacing the previous part in the browser display. The HTTP connection is kept open as long as the server has more parts to deliver – it can be held open indefinitely if required. The word 'part' may be a bit misleading here – the server is treating the communication as one very long 'web page' delivered in a series of parts, but since each part completely replaces the previous part on-screen, it is really more like a series of web pages. This differs from a normal web page where nothing is displayed by the browser until the whole page has been received. Here is a simple example showing both the HTTP headers and the HTTP body of the response that the client would receive.

```
Content-type: multipart/x-mixed-replace; boundary=PartBoundary
--PartBoundary
Content-type: text/HTML
Content-length: 55
<HTML><HEAD></HEAD><BODY> <H3>Hello</H3> </BODY></HTML>
--PartBoundary
Content-type: text/HTML
Content-length: 61
<HTML><HEAD></HEAD><BODY> <H2>Hello again</H2> </BODY></HTML>
--PartBoundary
Content-type: text/HTML
Content-length: 67
<HTML><HEAD></HEAD><BODY> <H1>Hello again again</H1> </BODY></HTML>
--PartBoundary--
```

This divides the web page into a series of parts with the boundaries indicated by a string specified by the `boundary` attribute shown on the first line, in this case the string `PartBoundary` – any string, however random, is acceptable here. The `boundary` string is preceded by two hyphens for all boundaries in the web page. The last boundary, indicating the end of the last part, is distinguished by also having two hyphens appended to the `boundary` string. After the last part has been sent, the server will close the HTTP connection.

Viewing this page with a suitable browser would show a series of three web pages (or parts) with hello messages, increasing in length and in font size each time, as illustrated in Figure 15.



Figure 15 Three successive views of the multipart content from the example above

Note that although we have shown three separate browser windows above (one for each part), in practice, there would be only one browser window and the content of each new part overwrites any previous content. The delay between the parts being sent from the server would be determined by the programming of the servlet or JSP page that generated the web page, as well as any network delays.

Note that we still have the usual sort of `Content-type` description at the start of each part – the content type in each part can be different but, when used for server push in this way, it will normally be the same for each part. For example, this multipart approach has been used for animations by sending a series of images each with content-type of some image format, or with longer delays it can be suitable for displaying webcam images.

Clearly this multipart approach could also be used for a more complex web page, such as a display of the latest bids in an auction. This page could be based on data extracted from a database with a new 'part page' sent to the client at any time that the server became aware of a change in the data. The whole web page is replaced even if the change to the display is rather minor, but this is true of the client polling and RSS approaches also – none of these can be used for selective updating. A problem particular to this approach is that the server might get locked into maintaining connections and sending repeated updates to faulty or malicious clients – so some sort of time limitation on the connection is normally imposed by the server.

Although multipart documents are an internet standard (see Subsection 2.4), the `x-mixed-replace` variety is not currently an internet standard (the `x` in `x-mixed-replace` indicates an experimental proposal). It is supported by some browsers, but not by all. We will not consider it in further detail in this course.

`x-mixed-replace` is currently not supported by Microsoft browsers.

SAQ 3

What are the advantages and disadvantages of each of the following?

- (a) Client polling
- (b) RSS
- (c) The server push approach using `multipart/x-mixed-replace`

ANSWER.....

- (a) Client polling (using the refresh `META` tag) is simple and works on all browsers. Its disadvantage is that it can generate considerable web traffic by frequent reloading of web pages. It also requires opening and closing connections to the server for every request, and many of the reloads may be unnecessary.
 - (b) RSS feeds eliminate the problem of unnecessarily downloading whole web pages even if nothing on that page has changed – the page is download only if something has changed since the last time it was checked. However, repeated polling of the server is required by the RSS reader, so the overhead of repeated connection opening and closing is similar to the case of client polling.
 - (c) The server push approach using `multipart/x-mixed-replace` puts the server in control of when the web page is reloaded. Hence page reloads should happen only when there is something new to download. Because the server keeps the HTTP connection with the client open while downloading a series of parts, it avoids the overhead of repeated opening and closing of HTTP connections. However, keeping the HTTP connection open may be a problem if there are many clients using this approach as most servers limit the number of possible HTTP connections. This approach is not supported by some browsers.
-

5.4 Other push technologies

We have seen that there are two basic requirements for push technology.

- ▶ Clients must register their interest in updates from the server.
- ▶ The server must send updates to all registered clients when a significant event occurs.

In *Unit 5* we discussed the publish/subscribe messaging model, which is similar in concept, but rather more general. In particular, in the *Unit 5* example, both clients and server could subscribe for particular types of messages, with all the messaging being handled by the middleware.

Initially we will look at a simpler approach where only the client registers an interest in updates ('subscribes') and where the client and server themselves handle the communication.

In the previous subsection we outlined a simple push approach that works for suitable web browser clients by using `multipart/x-mixed-replace`. It does, however, still rely on replacing the whole content of the browser window when an update is required. This can make for poor responsiveness and a rather jerky user interface. In this subsection we look at other approaches that allow for a more focused communication between the client and the server. Such communication requires a more sophisticated client, normally either an applet client or an application client (a full Java application). It usually makes very little, if any, difference to the server whether it is dealing with an applet client or an application client.

There are several possibilities for how servers may communicate with applets and application clients, as follows:

- ▶ UDP
- ▶ multicast UDP
- ▶ TCP/IP
- ▶ messaging service.

We will consider each of these in turn.

5.5 Push using UDP

You should recall that UDP (User Datagram Protocol) is the main connectionless protocol used on the internet. Because it does not require the setting up of a connection between the sender and receiver of a message, it should be more efficient than, for example, TCP, which requires such a connection. On the other hand, UDP does not guarantee that message data packets will arrive in the right order or even that they will arrive at all. Unreliability is the price to pay for this faster delivery of messages, and especially for some time-sensitive applications, such as multimedia streams, this trade-off makes sense.

This lack of the need to set up a connection makes UDP a useful choice for efficient communication between client and server.

When each client is created and initialised, it sends the server a UDP datagram (a message) – it must know the IP address and port number for the server. A UDP datagram contains the IP address of the sender, so the server can then store the IP address for this client in a list of IP addresses for all the registered clients (Figure 16).

We expect you to know the basic facts about UDP from prior study – for example, of M254 or M257.

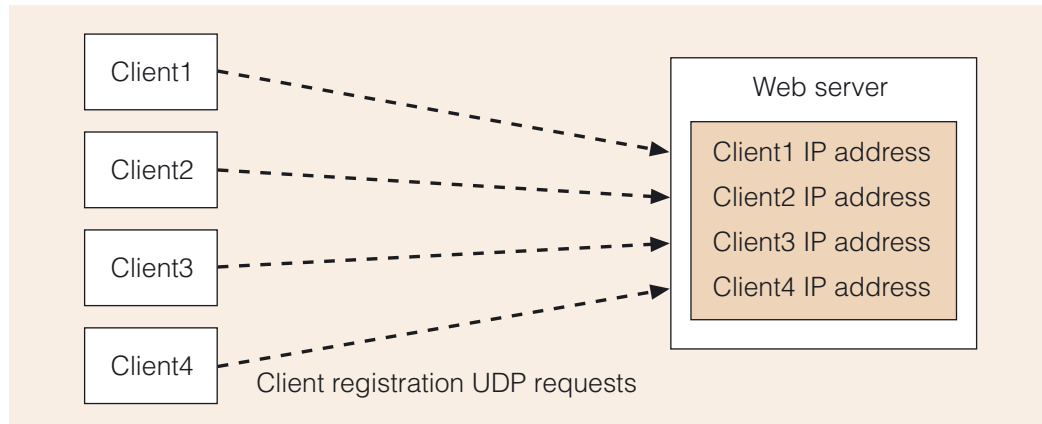


Figure 16 UDP push – clients registering their IP addresses with the server

The server will acknowledge that the client has registered and will send the required data for the client to display. If any change occurs in the data required by clients, the server must work through the list of registered clients and send a UDP message, containing the new data, to each client. Clients can then update their displays so that users are kept up to date (Figure 17).

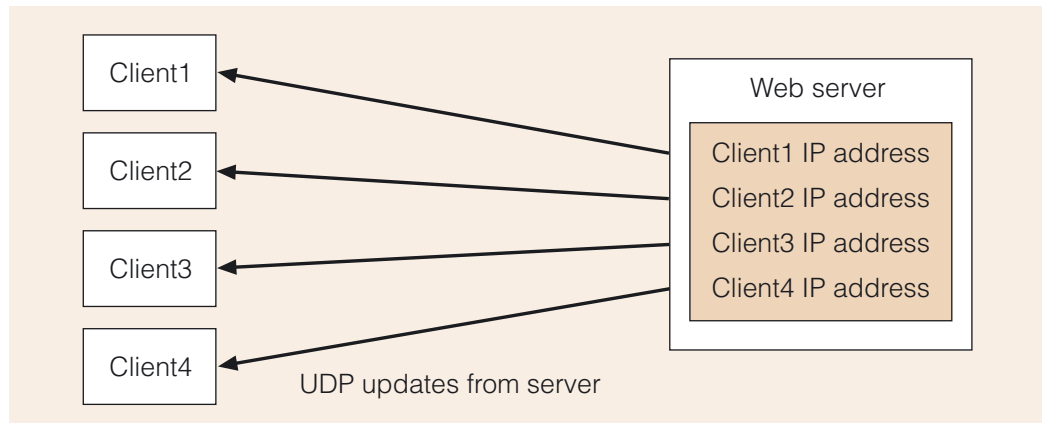


Figure 17 UDP push – server sending a UDP message to update each registered client

Consider the example of an online auction, where bids are submitted to the server by clients. Without push technology, only the bidding client would know about their bid, until the next time the other clients requested an update. With a push approach, all other clients can be informed of any new bids, the current highest bid and so on, almost as soon as any change happens.

We have seen that the relative efficiency of UDP communication is an advantage of this approach, although the nature of the protocol means that some updates to clients might be lost in transit. Another limitation is that UDP datagrams have a limited maximum size – normally a maximum of 65 527 bytes of data. Furthermore, if there are many clients registered, there is a considerable overhead for the server in sending a message to each client whenever a significant event occurs – this could be particularly serious in the closing stages of an auction when typically many people are trying to submit bids within a short time. This last problem can be addressed using a refinement of the UDP approach, known as UDP multicast.

5.6 Push using UDP multicast

We met the term **multicast** previously in *Unit 5* – recall that it refers to a form of communication whereby the sender sends a message once onto a network where it can

be read by many receivers. In multicasting, only receivers who have previously joined a special multicast group will receive the message. This differs from **broadcasting** where the message is sent to all hosts on a network; broadcasting is usually used only on local networks, to avoid flooding the world with messages. The multicasting network protocols take care of transmitting or replicating the message in as efficient a way as possible. This is clearly useful when a server needs to communicate the same message to many clients. UDP provides just such a multicast option. This can be used to refine the UDP approach discussed in the preceding subsection, which is technically known as **unicast** – one sender and one receiver at a time.

UDP multicast works by sending a message to an IP address within a certain range of addresses reserved for multicast messages. Hosts that wish to receive the multicast message must register to 'listen' for messages sent to that IP address. They do this by opening a special **multicast socket** attached to this address and then 'joining' a multicast group. Because multicast is a feature of the Internet Protocol (IP), this registration is with the network (Figure 18), rather than with a server as in the case of UDP unicast.

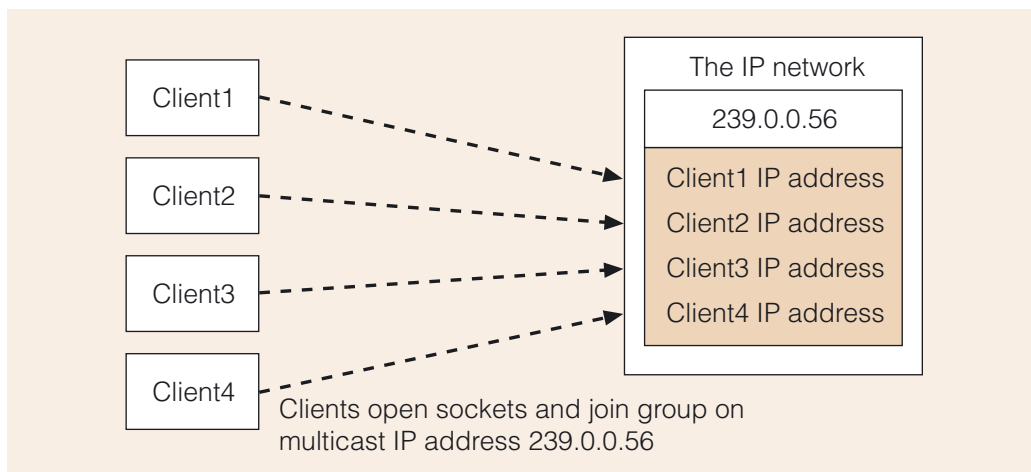


Figure 18 UDP multicast push – clients joining a multicast group

Sample Java code for the client setup is as follows.

```
InetAddress group = InetAddress.getByName("239.0.0.56");
MulticastSocket s = new MulticastSocket(2345); //use port 2345
s.joinGroup(group);
```

The `MulticastSocket` class is derived from the `DatagramSocket` class used for normal (unicast) UDP and provides additional methods for joining and leaving a multicast group. This socket can be used both for receiving and sending multicast UDP messages, but for server push the client needs only to receive, as in the following code, where `s` is the multicast socket opened in the previous code extract.

```
byte[] buffer = new byte[1024];
packet = new DatagramPacket(buffer, buffer.length);
s.receive(packet);
```

The socket on the server side can be a standard UDP socket (`DatagramSocket` class) or may be the special multicast socket we described above. To *send* to a multicast address it is not necessary to be a member of the multicast group. If the server wishes to *receive* a copy of its own multicast message or to receive multicast messages from other senders, then it would need to use a multicast socket and join a group, as described above. For our purposes of server push, this should not be necessary and a standard UDP socket will suffice on the server. The server simply sends normal UDP datagrams to the multicast address and port number that the client group uses (Figure 19).

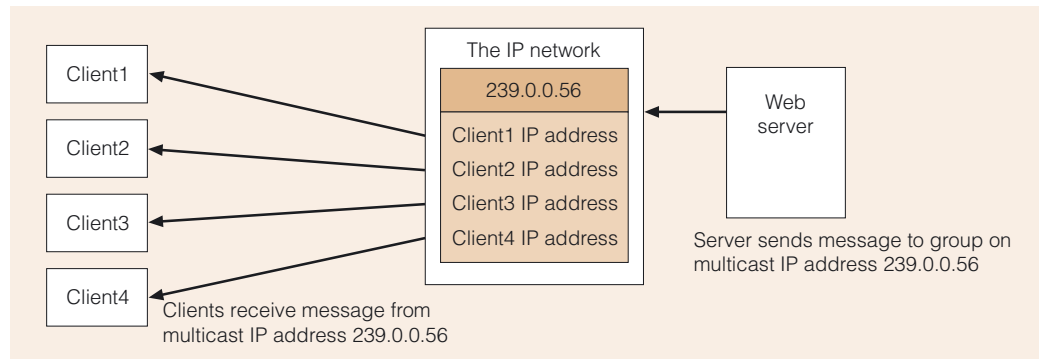


Figure 19 UDP multicast push – server sending an update to the client group

Sample Java code for the server to set up and send a UDP packet is as follows.

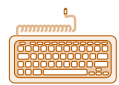
```
InetAddress group = InetAddress.getByName("239.0.0.56");
DatagramSocket s = new DatagramSocket();
...
byte[] buffer = new byte[1024];
packet = new DatagramPacket(buffer, buffer.length, group, 2345);
s.send(packet);
```

Note that the standard UDP socket is not linked to any particular IP address or port. These details are specified in the message itself (when the `DatagramPacket` object is created). We are using the same IP address and port number expected by the multicast group of clients.

IP addresses in the range 224.0.0.0 up to 239.255.255.255 are reserved for multicast use. Different subsets of these addresses are used, depending on whether the message is intended solely for clients within the local network or for clients more widely distributed and requiring messages to pass across routers.

An important limitation is that multicast messages are defined with a maximum 'time-to-live', a number initially in the range 0 to 255, which is decremented every time the message passes through a router. Once the time-to-live reaches zero, the message will no longer be passed on by the routers. Together with the different range of addresses for use in local networks, this ensures that multicast messages do not live forever and spread everywhere like a sort of spam.

Not all network routers support the necessary protocols to make UDP multicast work. Therefore it is best suited to intranet applications. An additional limitation is that at present Java applets are not allowed to use UDP multicasting.



Activity 9.5

UDP push for auction bids.

5.7 Push using TCP/IP

Recall that TCP is the standard connection-oriented protocol in use on the internet. Its main advantage over UDP is that it is more reliable – data packets that are corrupt or lost are automatically sent again and any packets received in the wrong order are reordered by the receiving host. Its disadvantage is the required overhead in processing, storage and time to provide this reliability.

Nevertheless, there may be situations in which it is appropriate to use TCP rather than UDP for pushing messages from the server to an applet or application client. In particular, if the number of clients, the likely network loading and the frequency of updates make it feasible to accept the TCP overhead, then TCP will avoid the problems of possible message loss that can occur with UDP.

If occasional loss of updates is acceptable, as long as another update succeeds soon after, then UDP is normally good enough.

The general approach to push using TCP/IP is similar to that for unicast UDP, discussed earlier. The important difference is that there is a TCP connection between the server and each client, and that this must be kept open for as long as the client is still active. We will not go into further details here, except to note that there is no TCP equivalent to multicast UDP and so no easy way of limiting the number of open connections.

Exercise 2

Why do you think there is no such thing as multicast TCP, while there is multicast UDP?

Discussion.....

In multicast UDP the server does not know anything about the clients in the multicast group it is sending to. It knows only the special IP address for the multicast group. This is acceptable in UDP because there is no need to set up a connection between server and client(s). In TCP, both the client and the server must know each other's IP addresses in order to set up a connection. This would negate one of the advantages of multicast over unicast – the multicast server does not need to keep a list of registered clients.

5.8 Push using a messaging service

In Subsection 8.3 of *Unit 5* we mentioned the Java Message Service (JMS) as an example of the publish/subscribe messaging model. This is a very generally applicable paradigm – in fact whole distributed systems are built around message-oriented middleware (MOM). For example, the Java EE business tier has a type of EJB component known as a **message bean**, which can be used to implement a message-oriented communication approach between Java EE components. These have a number of potential advantages in providing a more loosely coupled approach than we have seen elsewhere, but a detailed account of EJB message beans is outside the scope of this course.

Now we consider how messaging can be used to implement server push. As noted above, the messaging service paradigm is more general than our requirements at this stage. In general, any component of the system may publish, subscribe or notify.

- ▶ *Publishing* involves announcing what types of event may occur.
- ▶ *Subscribing* means registering an interest in one or more of the published types of event.
- ▶ *Notification* is the process of informing subscribers to a particular type of event that such an event has occurred.

For our purposes, considering The Music Store auction example, only the server can publish or notify, and clients can only subscribe. JMS is one of many MOM products from different vendors – we use this as our example system here as it is a standard part of Java EE. JMS can be used either for the message queue system approach, outlined in *Unit 5*, Subsection 8.2, or for the publish/subscribe approach. In this case we are interested only in the latter.

The publishing process involves setting up one or more **topics** – each one represents an area of interest that clients can subscribe to. In our case the only topic is the detail of the latest auction bid. Clients must subscribe to this topic as soon as they start running – they do this by communicating with the messaging system, rather than directly with the server. This process is illustrated in Figure 20.

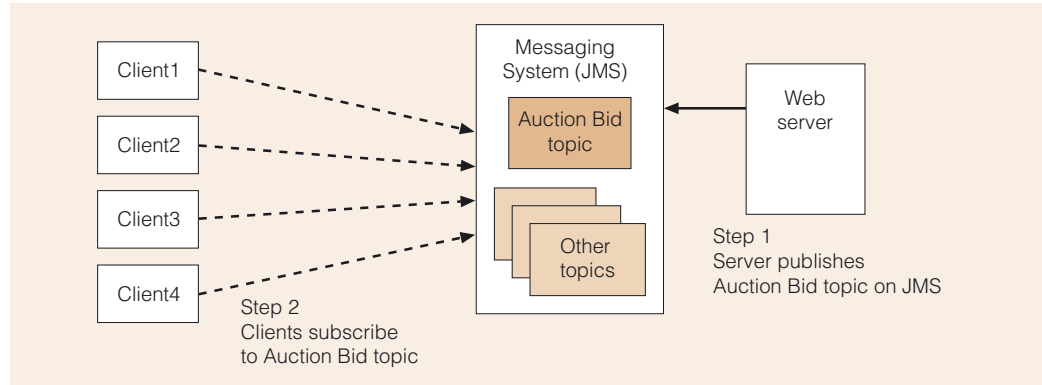


Figure 20 Messaging system (JMS) using publish/subscribe for server push

Clients can send auction bids directly to the server using the web tier or directly to the business tier as we have seen before. When the server receives a new bid, it notifies all clients of the bid details, again indirectly via the messaging service, as shown in Figure 21.

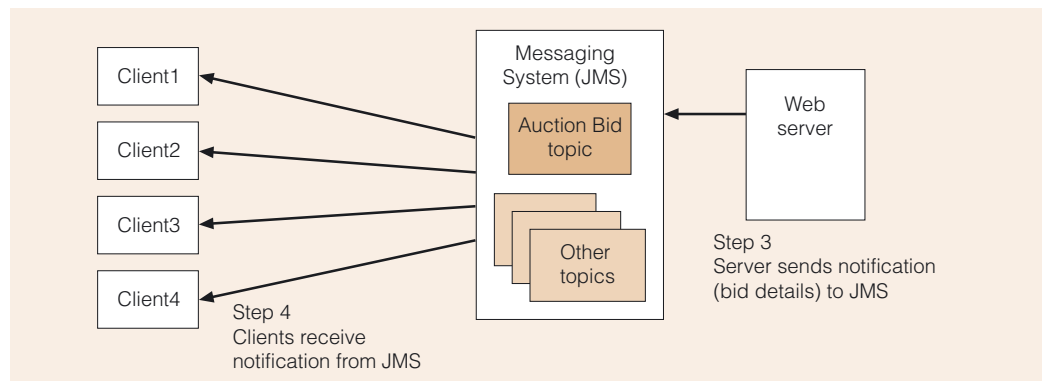


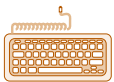
Figure 21 Server notifies subscribing clients of the latest auction bid

This means that the server does not need to keep track of the individual clients. In this sense the messaging service is similar to the UDP multicast approach, but has the major advantage of being reliable. Any message that cannot be delivered to a client immediately will be retained by the messaging service so that it can try again as many times as the system configuration requires.

TCP communication is also reliable, of course, but has the potential disadvantage that it is synchronous – the server in a TCP communication with the client has to wait for a confirmation of successful receipt before it can proceed with other activities.

Asynchronous messaging allows the sender of a message to deposit the message with the middleware and then proceed to other activities – this can help maintain the responsiveness of the system.

In practice, the JMS runs as part of the Java EE server. We have set up an activity so that you can try it out.



Activity 9.6

JMS messaging for auction bids.

SAQ 4

What are the pros and cons of the following push technologies for applets and application clients?

- (a) UDP
- (b) Multicast UDP
- (c) TCP
- (d) Messaging services

ANSWER.....

- (a) UDP is connectionless and therefore each message is relatively efficient. However, it is unreliable in that messages can be lost or corrupted. It also has the disadvantage for large-scale systems that it requires the server to store IP addresses of all registered clients and to send one message per registered client for each notification.
 - (b) Multicast UDP has the same pros and cons as normal (unicast) UDP, but is more efficient when the sender has to communicate the same message to many receivers. A disadvantage is that not all routers support multicast UDP.
 - (c) TCP is a reliable connection-oriented protocol in that lost, corrupt or out-of-order data packets will be restored automatically by the TCP protocol software. Its disadvantage is relative inefficiency – setting up connections takes significant time and effort, and means that it is not suited to multicasting.
 - (d) Messaging services are the most general approach. They use asynchronous communication, which can aid responsiveness, combined with reliability. The publish/subscribe approach is well suited to implementing server push. However, the middleware may introduce a significant performance overhead in some cases, and most middleware is proprietary rather than standard.
-

6

Time in distributed systems

In this section we examine the need for time and why it requires special consideration in distributed systems. We also explain what this has to do with our example of The Music Store online auction.

6.1 The problem of time in computer systems

Almost all computer systems have some indication of the current time and they use this for a variety of purposes; for example, the following.

- ▶ They may need to carry out actions at a specific time, or after a specified delay, especially in real-time systems.
- ▶ They may use time-stamps to record when something was done – this can be useful in managing transactions so that actions can be undone in the correct order if a transaction has failed.
- ▶ Systems normally record the time at which a file was created and last modified – this has a particular use by programs such as `make` or `ant` that allow selective compilation and building of executable programs after parts of the source code of a large system have been changed.
- ▶ Time is used to synchronise actions across large, widely distributed systems such as aerospace or military applications. A highly accurate value of the current time is used by the satellite-based Global Positioning System (GPS) to allow calculation of accurate positions of places on the Earth.

The `ant` program is used, for example, by NetBeans; `make` is an older tool with similar functions.

Time is very important in online auctions, such as when bidding for instruments in The Music Store. The auction for each item has a precise finishing time and the highest bid received by that time secures the item. As we will see, there are special so-called **sniping** programs which will attempt to make a bid as late as possible in order to secure auction items for the least increase over the previous highest bid. These programs depend crucially on having an accurate value of the auction closing time.

For a centralised computer system, providing an accurate indication of the current time requires the ability to set the correct time initially and then some means of maintaining that time accurately. Most modern computer systems have a hardware clock, usually based on a vibrating quartz crystal. The computer operating system uses the 'tick rate' maintained by this hardware clock to regularly update a software clock value. However, even if the time is set very accurately initially, these simple crystal-based clocks will tend to run noticeably fast or slow or even erratically – this deviation from 'perfect time' is known as **drift**. The **drift rate** is the deviation from perfect time per unit of time (usually per second). Typical computer hardware clocks have a drift rate of between 10^{-5} and 10^{-6} seconds per second, meaning that at best they would differ from perfect time by 1 second every 10^6 seconds (11.6 days). In practice, many PC clocks are much less accurate than this. The drift is affected by factors in the physical environment such as temperature, pressure or magnetic fields because these influence the frequency of oscillation of the crystal.

We explain below what we mean by perfect time.

In distributed systems, there is typically a **local clock** for each host in the system. Since each of these local clocks will drift, but usually not at the same rate, each different computer will tend to have a slightly different time value. The difference in clock readings between any two hosts is known as their **clock skew**. You might think that this

problem could be resolved by the hosts comparing times and resetting their time to the correct time – but which host has the correct time? It turns out that for a distributed system it is very difficult to achieve an accurate **global time**, a time that all hosts agree on. The implication of this is that it is hard to be sure of the precise order of events that occur in different hosts, and this results in difficulties with approaches like time-stamped events in transactions.

Before we consider this problem and its remedies in more detail, we explain where we get the perfect time that we have mentioned in the above discussion.

6.2 Perfect time

Once upon a time, our measurement of time into years, months, days, hours and seconds was based on astronomical phenomena. The year was defined by the period of the Earth's orbit around the Sun, and the day was defined by the period of rotation of the Earth. The day was then subdivided into the smaller units of hours, minutes and seconds. These definitions have long been much too inaccurate for scientific use. Moreover, the astronomical day is getting steadily longer as the Earth's rotation gradually slows down, mainly due to tidal friction. Incidentally, the energy lost by the Earth is transferred to the moon, which is gradually orbiting faster and moving further away. This is nothing to worry about too much, but it has cumulative effects over many centuries and millennia. The change in day length is noticeable, for example, by scientists who study ancient fossils.

The day is gaining about 1.7 milliseconds per century, and the moon is moving away at less than 4 cm per year.



Tidal acceleration.

Days are getting longer

Researchers examining ancient corals noted that annual growth patterns suggested [that] there were more days in a year in fossil corals from the Devonian Period (380 million years ago) [which] recorded 400 daily cycles. About 290 million years ago in the Pennsylvanian Period, there were 390 daily cycles each year. Assuming that Earth's revolution around the Sun has not changed dramatically, this means that the number of hours per day has been increasing and that Earth's rotation has been slowing. Today, the length of a day is 24 hours. During the Pennsylvanian Period a day was about 22.4 hours long. In the Devonian Period, a day was approximately 21.8 hours long. Earth's rotation appears to be slowing at a rate of about 2 seconds every 100,000 years.

(Lunar and Planetary Institute, 2004)

Because of these unsatisfactory properties of astronomical time, since 1967 the official measurement of time has been carried out using extremely accurate **atomic clocks**, based on the element caesium. The second is now defined as the time taken for 9 192 631 770 oscillations of the caesium atom's resonant frequency. The world's official time is kept by caesium atomic clocks at a number of national standards laboratories around the world, such as the National Physical Laboratory (NPL) in the UK and the National Institute of Standards and Technology (NIST) in the USA (Figure 22).



Atomic clocks.



Figure 22 The NIST-F1 atomic clock, which began operation at NIST in 1999, has an uncertainty of 1.7×10^{-15} , i.e. an accuracy of about 1 second in 20 million years, making it one of the most accurate clocks ever made

GMT, the time used in the UK and Ireland in the winter, is actually the same as UTC.

The international standard time based on these atomic clocks is known as **Coordinated Universal Time (UTC)**, the acronym being based on a compromise between its English language and French language names. It is actually calculated as an average of the values generated by the standards laboratories' caesium clocks. The rotation of the Earth, which determines astronomical time, runs slower than UTC at about 2 milliseconds per day. To ensure that UTC does not deviate too far from astronomical time, by international agreement a 'leap' second is added to UTC occasionally – normally about every 12 to 18 months. We cannot change the rotation of the Earth, but we can change UTC – this ensures that it stays synchronised with astronomical time. The UTC time value is accessible, via radio signals broadcast from institutes such as NIST and NPL, from satellites, including GPS satellites, and also through telephone lines.

Hence a computer can be closely synchronised to UTC by means of a radio receiver or satellite receiver linked directly to the computer's system. Such atomic clock radio receivers are, however, expensive. It is not currently feasible to fit such receivers to most computers, nor is this extreme accuracy necessary for most systems. Hence, many systems rely for accurate time on access over a network to a computer with an atomic clock radio. This 'second-hand' time value is likely to be less accurate than directly accessing time from a radio or a satellite, but sufficient for many purposes. A number of protocols have been defined to control this dissemination of time values across networks, including the internet, for which the most important is the Network Time Protocol (NTP), discussed below.

6.3 Synchronising across a distributed system

To keep a computer clock accurate, it must be regularly **synchronised** by comparing it with one or more other clocks. In the simplest case, we compare the computer clock with a more accurate time source, such as an atomic radio clock, or a time server (see below), and make the appropriate correction. If the more accurate time source provides standard time, such as a close approximation to UTC, this is known as **external synchronisation**.

In some situations a standard clock may not be readily available. In these cases it may be possible for a host to compare its clock value with the clocks of other hosts in the same system so that all hosts can ensure that they have approximately the same clock time. This clock time may differ significantly from the correct time shown by UTC, but for some applications this may not matter. This is known as **internal synchronisation**. A common approach to internal synchronisation is the Berkeley algorithm (Gusella and Zatti, 1989), which is primarily intended for use in local area networks (LANs). We will not consider this algorithm further in this course.

It is usually not advisable to synchronise by decreasing the value of a clock that is too fast, as this could result in events being recorded as occurring at times earlier than their causes. To overcome this potential problem with **causality** – the relationship between an event and its effect – we can adjust a fast clock by temporarily slowing down the normal increase of the clock value until it is back in synchronisation with the more accurate reference clock. This ensures that clock time is monotonic, in other words, it never goes backwards. If a clock is slow, there are no such problems with causality and we can safely adjust it to the correct time.

Exercise 3

Can you think of situations or applications where each of the following types of clock synchronisation would be appropriate in distributed systems?

- (a) External synchronisation
- (b) Internal synchronisation

Discussion.....

- (a) This is essential where the time used by hosts is required to be correct in relation to standard time such as UTC. This would include applications that must carry out an activity at a specific actual time, such as the finishing time of an internet auction.
- (b) Internal synchronisation is acceptable when the system time does not need to reflect the real-world time accurately. Such applications might be those where time-stamping is used to indicate the order in which events occurred (such as in transactions). Here the actual values of the time are not important, only their relative order.

There is a further issue that we need to consider in respect of setting time across distributed systems. Suppose that some master computer on a given network is linked directly to an atomic radio clock. This master computer can read the radio signal, which typically arrives once per second, and reset its clock as often as necessary to ensure it is correct. You might think that this master computer can just send the accurate time value regularly to other hosts in the network. Unfortunately, this does not take account of the significant time delay in the arrival of messages sent across large-scale, distributed systems. For example, packets of data sent across the internet may take a significant fraction of a second to travel to their destination. If we knew that the message would always take, say, 100 milliseconds to travel from the master computer to a certain host, then we could allow for this in adjusting the host computer clock. However, in most distributed systems the message delay is uncertain and is affected by many factors, such as network loading, message routing, transient faults, long-term faults and so on.



Activity 9.7

Measuring round-trip times using the ping program.

The Network Time Protocol

Given the above discussion, it is clear that only computers with direct access to a source of UTC time, such as a radio clock or satellite receiver, can be kept in close synchronisation with standard time (within 1 millisecond). Other computers within a distributed system without direct UTC sources can synchronise with the master



Network Time Protocol.

computer, but this process will lose some accuracy because of the uncertainty introduced by variable network delay times.

The most commonly used approach on the internet for dealing with clock synchronisation is called the Network Time Protocol (NTP). There is also a variant known as Simple Network Time Protocol (SNTP) that uses simpler algorithms but provides less precision than NTP. The NTP service has a number of computers, known as **primary time servers**, that have direct access to a standard time source (such as an atomic radio clock). The system is hierarchical with a series of numbered layers or strata. The atomic radio clocks or other standard time sources are, by convention, considered as stratum 0. Then come the primary servers at stratum 1, which provide time values for secondary servers at stratum 2, and so on through computers at strata 3, 4, ..., with decreasing accuracy at higher-stratum numbers. For example, the eBay auction site uses stratum 1 time. Note that stratum 0 time sources are directly connected (physically adjacent) to the stratum 1 primary servers – all other connections to servers in other strata are via a network. See Figure 23 for an illustration of an NTP hierarchy.

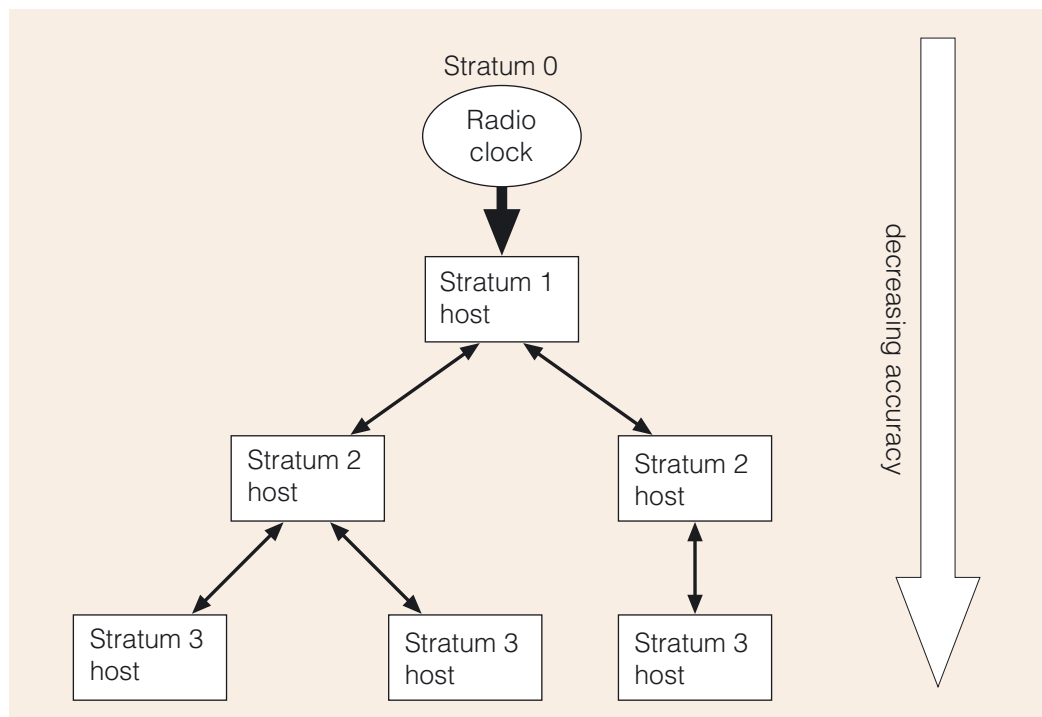


Figure 23 An example computer network in strata according to the NTP



NTP servers.

There are many NTP primary servers located all over the world. Most countries have at most a few primary servers, if any, and about half of the total number are in the USA. The NTP is very sophisticated and has different modes of operation for LANs, WANs and servers communicating within the same stratum. Each computer participating in an NTP network runs software to allow it to communicate with other computers in the higher- and lower-numbered strata and also in the same stratum, where appropriate.

A stratum 1 time server will typically have a time within 1 millisecond of UTC. Because of uncertain network delays, a stratum 2 time server will be expected to be only within 10–100 milliseconds of UTC. Each subsequent stratum adds an additional 10–100 milliseconds of inaccuracy.

The most important part of the system for internet purposes is the synchronisation between computers in different strata. This uses an approach known as Cristian's algorithm (Cristian, 1989) for **probabilistic synchronisation**. We consider the interchange between a primary and secondary NTP server, but in principle the same approach will work between any two strata.

Probabilistic here means that the synchronisation is not exact but is based on statistical assumptions.

The secondary server requests a time value from the primary server and measures the round-trip time for the request and response. On the often reasonable assumption that the outward and return trips take equal time, the secondary server sets its local clock to the time value received from the primary server plus half the round-trip time (see Figure 24).

To make some allowance for network variability, the secondary server may make several requests before using the minimum of these round-trip times in setting its local clock – this minimises the possible clock skew between the two hosts. Because of these statistical uncertainties, the secondary server time cannot be guaranteed to be as accurate as that of the primary server. The same issues apply to interactions at each level, so that the clocks at each successive higher-numbered stratum are less accurate than those at lower-numbered strata.

The algorithm is probabilistic in that the accuracy of synchronisation can be increased by increasing the number of synchronisation requests, but there is clearly a trade-off between accuracy and the number of such requests. The alternatives to probabilistic algorithms are deterministic synchronisation algorithms, which give a 100% certain guarantee of the maximum clock skew – probabilistic algorithms normally require less synchronisation requests and are more flexible since they can be adapted to the level of accuracy required.

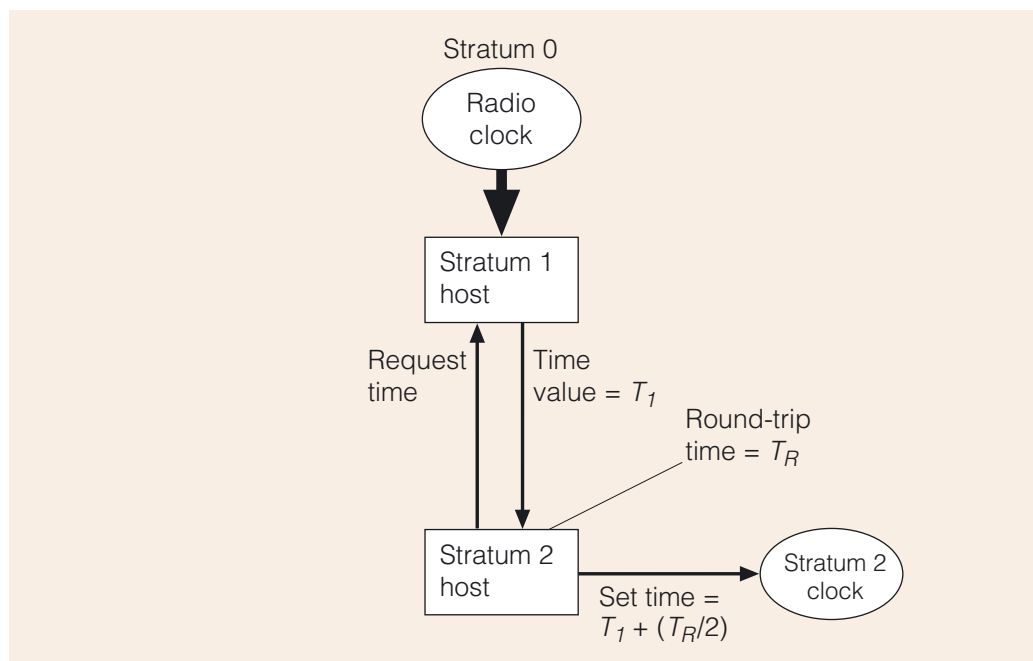


Figure 24 How NTP uses Cristian's algorithm to synchronise clocks

The architecture of the NTP provides redundancy to allow for possible failure – there are multiple servers available within each stratum and, via the internet, multiple communication routes. It is also designed to be scaleable to a large number of clients. The system has security features, such as authentication techniques to check that the time data has been sent by the correct host – this is to guard against both accidental and deliberate interference.

Exercise 4

A stratum 2 NTP host uses Cristian's algorithm to synchronise with a stratum 1 host. It sends a number of requests, as described above, to estimate the round-trip time for messages between the two hosts. The measured round-trip times are 34, 26 and 30 milliseconds. The successive time values received, T_1 , T_2 and T_3 , respectively, are 10:45:00:000, 10:45:00:049 and 10:45:00:100, using the format *hh:mm:ss:uuu* (where *uuu* represents the milliseconds).

- What value would the stratum 2 host set if it uses only the first time T_1 and the corresponding round-trip time?
- What value would it set in order to get the most accurate value obtainable from this data?

Discussion.....

- The first round-trip time T_R is 34 milliseconds. We need to set the time to $T_1 + T_R/2$, which is T_1 plus 17 milliseconds. Hence the time should be set to 10:45:00:017.
- The lowest round-trip time T_R is 26 milliseconds, and this gives the most accurate possible time setting from this data. We need to set the time to $T_2 + T_R/2$ which is T_2 plus 13 milliseconds. Hence the time should be set to 10:45:00:062. Note that we do not average the round-trip times in this approach.

6.4 Logical time

For some applications of time in distributed systems, we are interested not in the actual clock time, but only in being able to use time to record the order of events. For example, when a message is sent across a network, from Host A to Host B, the time of arrival of the message should be later than the time at which the message was sent. In a distributed system, where each host has its own local clock, if these are not regularly synchronised it is perfectly possible that this 'obvious' requirement will not hold – Host B may record the arrival time as *earlier* than the send time recorded by Host A.

One solution is the synchronisation approach exemplified in the previous subsection, where regular exchange of timing information is used to keep each host having similar time values within some acceptable limit. But this may require a considerable amount of traffic and the potential time error is also subject to the probabilistic effects noted by Cristian – which may be significant if message delays in the system are highly variable.

An alternative approach is for each host to use **logical time**. This is not closely related, if at all, to the actual time. However, logical time can be constructed so as to ensure that the order of events is consistent with causality – i.e. that the cause of an event (such as sending a message) occurs at an earlier logical time than the event itself (such as the arrival of the message).

A well-known approach to logical time in distributed systems is the Lamport time-stamp (Lamport, 1978). This is useful in controlling distributed transactions and in making it possible, if necessary, to undo a partially completed transaction.

The Lamport approach can be summarised as follows. For simplicity we consider events to be either sending or receiving a message, but the approach applies more generally. Each host has a logical clock which it increments by one unit in only two situations:

- ▶ (always) immediately after sending a message;
- ▶ (possibly) after receiving a message.

This guarantees that all events that occur on the same host, such as one process sending a message to another process, will follow causality – the logical time of sending a message will be earlier than the logical time of the message's arrival. If a message is sent to another host, it is also **time-stamped** – the logical time at the sending host is included along with the message data. The receiving host then compares the message time-stamp with its current logical time and, if necessary, increments its own logical time to ensure that the arrival time is later than the sending time. If the receiving host's logical time is already later than the sender's time-stamp, then the receiver does not need to adjust its logical clock.

This scheme ensures that all events are time-stamped in such a way that an effect always has a later time-stamp than its cause. The logical clocks may not bear any relation to standard clock time, but in this case that does not matter. Figure 25 shows a simple example.

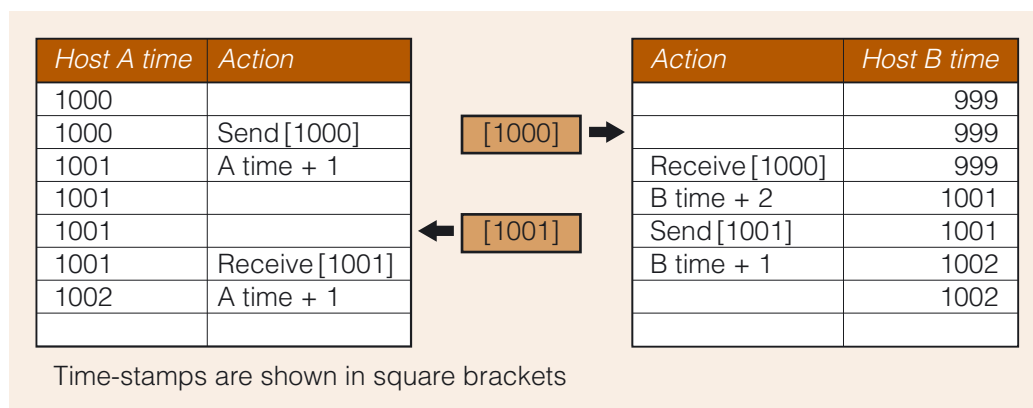


Figure 25 An example of logical clocks in Hosts A and B exchanging time-stamped messages

6.5 An example – auction time

A traditional auction takes place in a physical auction room, with bidders or their representatives physically present. The auction terminates when the auctioneer is clear that there are no further higher bids forthcoming from those present.

The rules for most online auctions are necessarily a little different. It would be very inconvenient for all the bidders for a given item, possibly in different time zones around the world, to stay online continuously from the start until the end of an auction. So online auctions normally have automatic bidding where the bidder indicates the maximum they are prepared to pay for an item and the auctioneer's software bids automatically on their behalf up to that maximum value. There is a precise closing time fixed in advance and the highest bidder at that time secures the item. This raises a number of issues about time in the vast distributed system that is the internet, some of which are fortunately fairly easy to resolve.

It is necessary to specify the exact second when the auction finishes, including the time zone or other standard used. The arrival time of a bid, rather than the time it was sent, determines whether it can be accepted. The bid may have been sent up to several seconds earlier (or much more in a case of a network fault).

The clock time at the auctioneer's server may differ by a little or by a lot from the clock time value at each of the potential thousands of bidding computers. As we have seen, clocks on ordinary PCs can be quite inaccurate and may easily be minutes away from the correct time. Hence it is important that the software clock on a bidder's computer is synchronised with the auctioneer's clock. On eBay, for example, the auction servers use stratum 1 time, which should be accurate to within 1 millisecond of UTC.

Auction sniping

It is possible for bidders on online auction sites to raise the maximum amount they are prepared to bid, as long as the auction has not yet reached its finish time. This normally occurs when a bidder periodically checks the auction site, sees that the bidding for an item now exceeds their maximum and perhaps decides that they are willing to pay more than they had decided previously.

This facility can be exploited by a process known as **auction sniping** – potential bidders check the auction for an item just before it closes and submit a bid that is higher than the previous highest bid. This leaves very little time for any other bidders to respond and so the sniper may well secure the item at a lower price than if there were time for further bidding from others. Clearly sniping requires that the bidder has a clock that is closely synchronised to the auctioneer's clock. There are two main possibilities. Assuming that the auctioneer uses a stratum 1 time server, the bidder may use at least a stratum 2 clock to make their system closely aligned to the time used by the auction. Alternatively, the bidder may synchronise (using, for example, NTP) regularly with the auction time, especially just before the sniping bid is placed. In either case, the bidder must allow for network delays in deciding the sending time, since the bid must arrive by the closing time.

Naturally, this process of sniping has now been automated and there are many websites that offer sniping services for popular online auction sites. You enter details of the item and the maximum you are prepared to pay and the sniping software does the rest. The synchronisation and the message delay estimation are now taken care of by the sniping website, rather than directly by the bidder. Usually, there is a small fee for this service.

We should mention at this point that although sniping is not illegal, some people consider it to be a rather dubious practice and against the spirit of online auctions. Sniping site names like AuctionStealer say it all. We leave you to decide your own view, as in this section we are primarily concerned with the technical issues of timing. You might like to note that snipers can, of course, be sniped by other people, so there can be an advantage in being able to estimate very accurately the latest possible time to bid.



Auction sniping.

SAQ 5

In distributed systems, what is the difference between physically synchronised clocks and logical clocks?

ANSWER.....

Physically synchronised clocks on different hosts will hold time values that are the same to within some specified tolerance. If they are externally synchronised, this common time will be synchronised to some standard time such as UTC. If they are internally synchronised, then the common time may differ significantly from standard time, but in some applications only internal synchronisation is essential.

Logical clocks can be used in some situations where synchronisation is not required. Each host maintains a clock which it updates immediately after an event, such as sending or receiving a message. Messages sent to other hosts are time-stamped with the logical time of the sender. Receivers update their local logical clocks, if necessary, to ensure that events always have a later logical time than the event that caused them. Thus logical clocks can be used to record an order of events, although they are not applicable when the actual standard time is needed, such as in many real-time systems.

7

Case study

The current scenario in the case study is an online auction site as follows.

A music shop sells second-hand and rare musical instruments. The instruments are stored in a warehouse from where they can be shipped to customers who have purchased them. There is a server computer in the music shop so that staff can keep the catalogue updated. There is also a computer in the warehouse to help keep track of goods in and out, and of what is currently in the warehouse.

The server in the shop is accessible to the public via the internet. Customers register with the system before use, entering contact details and financial details so that they can pay for purchases. The payments handling is outsourced to another organisation and we do not consider this at present.

The shop system has software to facilitate auctions over the internet. Instead of a fixed price, each item has a starting price, a fixed date and time at which the auction finishes, and possibly a reserve price (the minimum acceptable price).

Customers place bids over the internet by indicating the highest price they are prepared to pay. The auction system automatically bids for them until it has reached that maximum price. It uses push technology to keep the customer in touch with bidding by other customers. The highest bid at the exact time the auction finishes secures the musical instrument. The system then accepts payment and authorises the warehouse to despatch the instrument.

This scenario is not quite the full system outlined in *Unit 1*, but it is sufficiently complex to be quite realistic and to allow us to see how it can employ many of the Java EE technologies that have been discussed in the last few units.



Activity 9.8
Online auction
scenario.

8

Summary

In this unit we have considered the various ways in which the client tier of a distributed system can be organised, with particular reference to Java EE. Thin clients are typically standard web browsers communicating via the HTTP protocol and these handle only display and communication with the server. The thin client is adequate for many systems but has a number of limitations. We looked in some detail at how thin clients interact with the server by means of HTTP methods and HTML forms, including use of hidden form fields.

More complex clients are known as thick clients, fat clients or rich clients and they may be used to provide more sophisticated user interfaces or to carry out significant processing on the client computer. These can be implemented using applets or stand-alone applications, which in Java EE are called application clients.

Thin clients and applet clients normally communicate with components of the Java EE web tier. Since application clients and, in some cases, applets, provide their own user interface, such systems can omit the web tier and communicate directly with the session beans in the business tier. For the application client, this can be achieved by running it in an application client container on the client machine. Alternatively the application client can be programmed to explicitly use JNDI to access session beans running on the server.

Thick clients allow more complex conversations between client and server so that forms of push technology can be used to allow the server to keep clients updated.

Finally we looked at the issue of time in distributed systems – distinguishing actual time from logical time – and noted the impossibility of perfect synchronisation between the hosts of a distributed system.

Figure 26 summarises the Java EE tiers and components, including three different types of containers that we discussed in *Units 6–9*. It also shows the main flow of communication between the tiers.

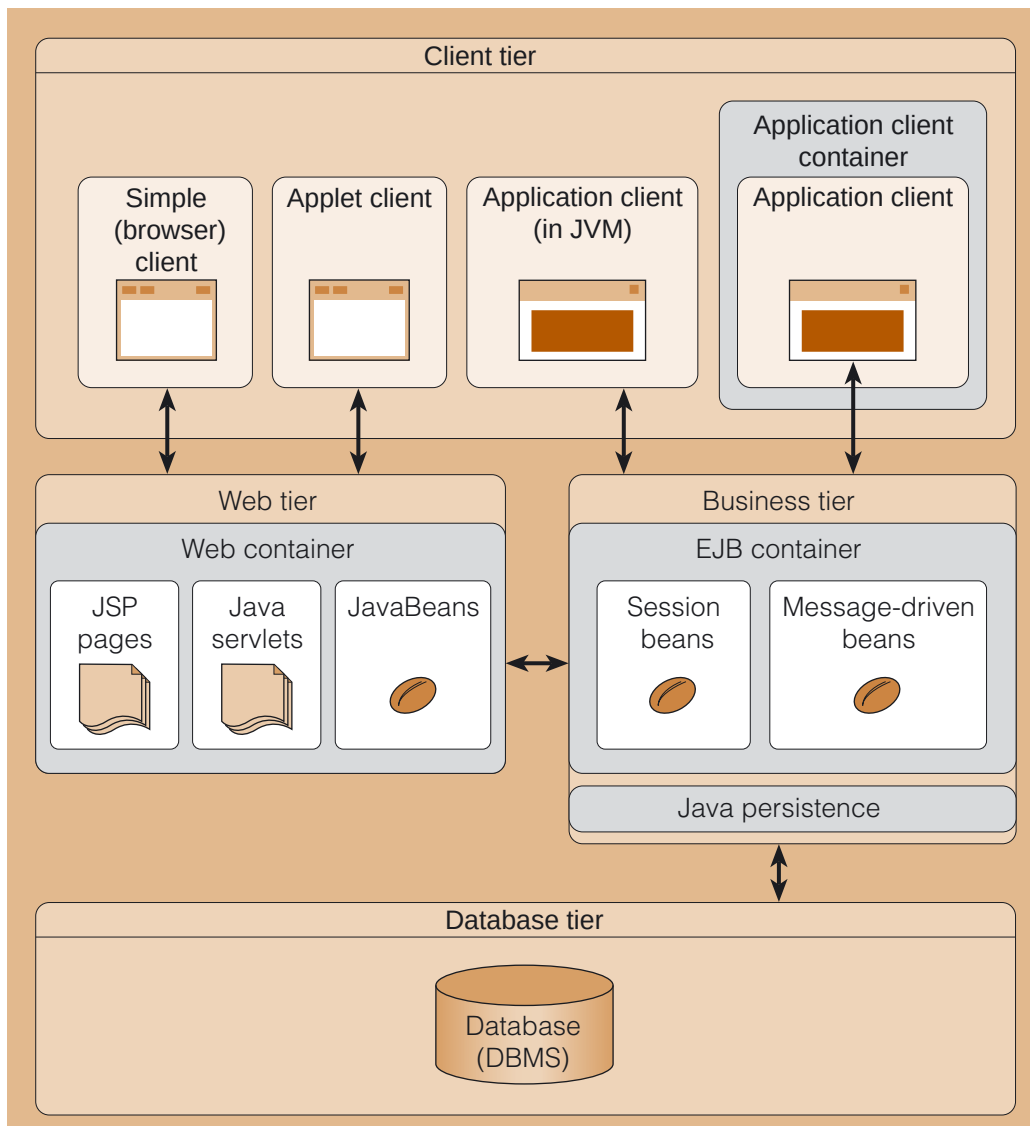


Figure 26 Java EE tiers, components and communication technologies

In the next unit we will look at the security issues for concurrent and distributed systems.

LEARNING OUTCOMES

When you have completed your study of this unit you should be able to do the following:

- ▶ explain how a browser can act as a thin client for a Java EE system;
- ▶ outline alternative approaches to client software, such as applets and applications;
- ▶ discuss the options for clients in communicating with the Java EE server;
- ▶ use thin client systems or develop fat client systems to interact with the middle tier;
- ▶ use a variety of server push technologies and understand their pros and cons;
- ▶ explain the issues concerning time in distributed systems;
- ▶ apply the concepts discussed in this unit to the auction system for The Music Store.

Glossary

applet A Java program that is downloaded along with a web page and run by the browser.

application A Java program that can run independently of a browser.

application client A Java EE client implemented as a Java application, running on the client computer either in its own JVM or in an application client container.

application client container A software environment for application clients that takes care of many of the routine requirements of communication, security and so on. In particular, it facilitates access from an application client to remote EJB session beans running on the server.

atomic clock An extremely accurate timekeeping device, based on the resonant frequency of atoms, usually of the element caesium, and on which official definitions for time units are now based.

attribute (HTML) A named item of additional data in an HTML tag. It is usually paired with a value, such as `method="POST"` or `type="HIDDEN"`, where `method` and `type` are attributes of the `<form>` tag and the `<INPUT>` tag respectively.

auction sniping A practice where potential bidders in a time-limited online auction access the auction for an item just before it closes and submit a bid that is higher than the current highest bid in the hope of securing the item before anyone else can raise their bid.

broadcasting Sending a message to all hosts on a network; normally limited to a local network.

causality The relationship between an event (a cause) and its effect – for our purposes we require a notion of time in a distributed system so that the causes occur at an earlier time than their effects.

client polling An automatic process that causes the browser to reload a web page at regular intervals by including special instructions in the HTML header of the web page.

clock skew The difference in clock readings between any two hosts in a distributed system.

Coordinated Universal Time (UTC) The international standard time based on atomic clocks.

drift The absolute deviation of a clock from perfect time (in seconds or a fraction of a second).

drift rate The clock deviation per unit of time (usually per second), such as typical computer hardware clocks' drift rate of between 10^{-5} and 10^{-6} seconds per second.

external synchronisation Keeping a clock accurate by regular comparisons with a more accurate external time source, such as a close approximation to UTC.

fat client See **thick client**.

global time A common value of the current time in a distributed system, in the sense that all hosts have perfectly synchronised clocks (an ideal that cannot be achieved).

hidden form field A parameter in a web form defined using an `<INPUT>` tag with the attribute value `type="HIDDEN"`, which means it is not displayed by the browser.

HTTP header The first part of an HTTP request or response preceding the body of the request or response. A response header includes information such as the number of bytes and the type of content returned, while the body typically contains web page content or other requested resource.

HTTPS A secure HTTP protocol that uses encryption to protect information sent across the Web.

internal synchronisation A process where a host compares its clock value with the clocks of other hosts in the same system so that all hosts can ensure they have approximately the same clock time, although this time may differ significantly from the correct (UTC) time.

Java Naming and Directory Interface (JNDI) An API for accessing naming and directory services in a vendor-independent way.

Java Plug-in Downloadable software that can be 'plugged in' to most browsers to provide a standard and up-to-date JVM and runtime environment enabling them to run applets using current versions of Java.

local clock The clock giving a time value in a particular host in a distributed system – typically local clocks show different values from each other.

logical time A system of recording time values so as to ensure that the order of events is consistent with causality – i.e. that the cause of an event (such as sending a message) occurs at an earlier logical time than the event itself (the arrival of the message). The time values are not normally related to the actual time shown on local or external clocks.

message bean A type of EJB component that can be used to implement a message-oriented communication approach between Java EE components.

middle tier The part of a layered distributed system that sits between the client tier and the EIS or database tier. In Java EE it may have two component tiers, the business tier and an optional web tier.

multicast A form of communication whereby the sender sends a message once onto a network where it can be read by many receivers who have previously registered to receive such messages.

multicast socket An extension of the standard Java socket concept, used to facilitate multicast communication.

persistent connection The approach used by HTTP v1.1 in keeping the TCP connection open after a first request to allow for possible subsequent requests. This contrasts with HTTP v1.0 which requires a new TCP connection for each request.

primary time server A host that has direct access to a standard time source (such as an atomic radio clock) and that makes this time value available to other hosts in a distributed system – in NTP, a stratum 1 host.

probabilistic synchronisation An approach to clock synchronisation in distributed systems that guarantees the accuracy of synchronisation with a certain probability as opposed to a deterministic (100% certain) guarantee – the accuracy can be increased by increasing the number of synchronisation requests.

pull technology An informal description of the request–response nature of HTTP – that information travels from the server to the client only when the client does something to ‘pull’ that information to it.

push technology An informal description of various ways to mitigate the request–response nature of HTTP by enabling the server to send (or ‘push’) information to the client, in order to keep the client up to date, without an explicit request from the client.

reserved characters Characters in a URL that have a special meaning, such as / or #, and so cannot be used as part of identifiers.

rich client See **thick client**.

RSS (Really Simple Syndication) A technology that allows users to sign up to receive updates from a website whenever something important has changed.

sandbox A restrictive environment in which untrusted Java applets execute. The word is derived from an American term for a children’s sandy playing area.

sniping See **auction sniping**.

synchronisation (time) To cause clocks to indicate the same time or time values as closely as possible.

thick client Complex client software that carries out significant processing on the client machine in a client–server system.

thin client In a client–server system, a simple client with very limited functionality, mainly limited to display and communication – for example, a web browser.

time-stamp A record of the time of an event (such as the sending of a message) that is attached to that event (such as being appended to the message in a data packet).

topic In a messaging system (such as JMS), a topic is a named area of interest to which clients can subscribe, such that they receive any messages relating to this area.

Transmission Control Protocol (TCP) connection A link agreed between two internet hosts that is established by a ‘handshake’ (an exchange of a number of initial data packets), prior to the hosts sending actual data.

unicast A form of communication involving one sender and one receiver at a time.

unsafe character One of the reserved characters or other characters that cannot be included in a URL without the possibility of confusion about their intended role or meaning.

URL encoding A means of including potentially unsafe characters in a URL by encoding them using the corresponding ASCII code, such as using %2F instead of /.

UTC See **Coordinated Universal Time**.

web service A standards-based web application that interact with other web applications for the purpose of providing a service.

References

- Cristian, F. (1989) 'Probabilistic clock synchronization', *Distributed Computing*, vol. 3, no. 3, pp. 146–58.
- Gusella, R. and Zatti, S. (1989) 'The accuracy of the clock synchronization achieved by TEMPO in Berkeley UNIX 4.3BSD', *IEEE Transactions on Software Engineering*, vol. 15, no. 7, pp. 847–53.
- Himelstein, L. and Siklos, R. (1999) *PointCast: The Rise and Fall of an Internet Star* [online], http://www.businessweek.com/1999/99_17/b3626167.htm (Accessed 10 September 2007).
- Lamport, L. (1978) 'Time, clocks, and the ordering of events in a distributed system', *Communications of the ACM*, vol. 21, no. 7, pp. 558–65.
- Louis, T. (1996) *ScreenSaver Newscast* [online], <http://www.tnl.net/who/bibliography/pointcast> (Accessed 1 February 2008).
- Lunar and Planetary Institute (2004) *Day and Night* [online], http://www.lpi.usra.edu/education/skytellers/day_night.shtml (Accessed 11 September 2007).
- Wikipedia (2007) *PointCast (dotcom)* [online], <http://en.wikipedia.org/wiki/Pointcast> (Accessed 10 September 2007).

Acknowledgements

Grateful acknowledgement is made to the following sources.

PointCast push technology (Subsection 5.1): Louis, T. (1996) *ScreenSaver Newscast*,
<http://www.tnl.net/who/bibliography/pointcast>

Figure 22: National Institute of Standards and Technology (Source:
<http://tf.nist.gov/cesium/atomichistory.htm>)

Index

A

- applet 6, 18–27
 - browser compatibility 26
 - lifecycle 22

application 6

- client 28
- client container 28

attribute (HTML) 11

auction sniping 46, 54

B

broadcasting 41

C

causality 49, 52

client

- applet 18–27
- application 28–32
- fat 6
- rich 6
- thick 6
- thin 6, 8–17

client polling 34

clock

- atomic 47
- local 46

clock skew 46

Coordinated Universal Time

see UTC

Cristian's algorithm 50

D

- drift 46
 - rate 46

H

hidden form field 14

HTTP header 9

HTTPS 13

J

Java Message Service (JMS)

see messaging service

Java Naming and Directory Interface (JNDI) 30

Java Plug-in 24

Java Web Start 31

M

message bean 43

messaging model, publish/subscribe 39, 43

messaging service 27, 43

- JMS 43
- topic 43

middle tier 6

multicast socket 41

multipart content type 36

N

Network Time Protocol (NTP) 50

P

persistent connection 16

primary time server 50

pull technology 17

push technology 17, 33–45

R

RSS (Really Simple Syndication) 33, 35

S

sandbox 24

sniping 46, 54

synchronisation (time) 48

- external 48
- internal 49
- probabilistic 50

T

TCP

- TCP connection 16
- push technology 42

time

- global 47
- logical 52

time-stamp 53

U

UDP 39

- multicast 40
- unicast 41

URL encoding 11

- reserved character 11
- unsafe character 11

UTC 48–49

W

web service 6